

```
/* ===== 01 文件: src/main.tsx 共 9 行 ===== */
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.tsx'
createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
/* ===== 02 文件: src/App.tsx 共 102 行 ===== */
import { useCallback, useState } from "react";
import { AnimatePresence, motion } from "framer-motion";
import { Brain } from "lucide-react";
import "./App.css";
import PhoneFrame from "./components/PhoneFrame";
import BottomNav from "./components/BottomNav";
import HomeScreen from "./screens/HomeScreen";
import TrainScreen from "./screens/TrainScreen";
import CommunityScreen from "./screens/CommunityScreen";
import DeviceScreen from "./screens/sub/DeviceScreen";
import PrescriptionScreen from "./screens/sub/PrescriptionScreen";
import PlayerScreen from "./screens/sub/PlayerScreen";
import AiChatScreen from "./screens/sub/AiChatScreen";
import GrowthScreen from "./screens/sub/GrowthScreen";
import RaceScreen from "./screens/sub/RaceScreen";
import { NavProvider, type SubRoute } from "./nav";
import type { SubName, TabKey } from "./data/mock";
import { RaceSessionProvider } from "./game-session";
import { useClickSound } from "./hooks/useClickSound";
import { ValenceLoopProvider } from "./state/useValenceLoop";
const SCREENS = {
  home: HomeScreen,
  train: TrainScreen,
  community: CommunityScreen,
} satisfies Record<TabKey, () => unknown>;
function renderSub(s: SubRoute) {
  const p = s.params ?? {};
  switch (s.name) {
    case "device": return <DeviceScreen />;
    case "prescription": return <PrescriptionScreen id={p.id as string | undefined} />;
    case "player": return <PlayerScreen from={p.from as string | undefined} />;
    case "aichat": return <AiChatScreen />;
    case "growth": return <GrowthScreen />;
    case "race": return <RaceScreen />;
  }
}
export default function App() {
  useClickSound();
  const [tab, setTab] = useState<TabKey>("home");
  const [sub, setSub] = useState<SubRoute | null>(null);
  const Screen = SCREENS[tab];
  const openSub = useCallback((name: SubName, params?: Record<string, unknown>) => setSub({ name, params }), []);
  const closeSub = useCallback(() => setSub(null), []);
```

```

const goTab = useCallback((t: TabKey) => { setSub(null); setTab(t); }, []);
const overlay = (
  <AnimatePresence>
    {sub && (
      <motion.div
        key={sub.name}
        className="phone-overlay"
        initial={{ x: "100%" }}
        animate={{ x: 0 }}
        exit={{ x: "100%" }}
        transition={{ type: "tween", duration: 0.32, ease: [0.22, 1, 0.36, 1] }}
      >
        {renderSub(sub)}
      </motion.div>
    )}
  </AnimatePresence>
);
return (
  <ValenceLoopProvider>
    <RaceSessionProvider>
      <NavProvider value={{ goTab, openSub, closeSub }}>
        <div className="app-shell">
          <aside className="app-aside">
            <div className="brand-row">
              <div className="brand-mark"><Brain size={20} /></div>
              <h1>智愈莘莘 NeuroHeal</h1>
            </div>
            <span className="kicker" style={{ marginTop: 6 }}>脑电情绪监测与干预反馈系统 • V1.0</span>
            <span className="tag">EEG 监测 • Valence 识别 • AI 反馈</span>
          </div>
          <p>
            连接便携式脑电头环，持续采集 EEG 脑电信号，识别 Valence 情绪效价，并由 AI 给出安抚或干预建议，形成实时反馈闭环。
          </p>
          <ul>
            <li>监测 • EEG 波形与 Valence 等级</li>
            <li>干预 • 呼吸训练 / 数字处方 / AI 陪伴 / 调节游戏</li>
            <li>记录 • 闭环记录与成长档案</li>
          </ul>
          <p style={{ fontSize: 12, color: "var(--t-tertiary)" }}>演示数据均为 mock • 用于说明实时闭环逻辑</p>
        </aside>
        <PhoneFrame nav={<BottomNav active={tab} onChange={setTab} />} overlay={overlay}>
          <AnimatePresence mode="wait">
            <motion.div
              key={tab}
              initial={{ opacity: 0, y: 10 }}
              animate={{ opacity: 1, y: 0 }}
              exit={{ opacity: 0, y: -8 }}
              transition={{ duration: 0.24, ease: [0.22, 1, 0.36, 1] }}
            >
              <Screen />
            </motion.div>
          </AnimatePresence>
        </PhoneFrame>
      </div>
    </NavProvider>
  </RaceSessionProvider>

```

```

    </ValenceLoopProvider>
  );
}
/* ===== 03 文件: src/nav.tsx 共 15 行 ===== */
import { createContext, useContext } from "react";
import type { SubName, TabKey } from "../data/mock";
export type SubRoute = { name: SubName; params?: Record<string, unknown> };
type NavCtx = {
  goTab: (t: TabKey) => void;
  openSub: (name: SubName, params?: Record<string, unknown>) => void;
  closeSub: () => void;
};
const Ctx = createContext<NavCtx | null>(null);
export const NavProvider = Ctx.Provider;
export function useNav(): NavCtx {
  const v = useContext(Ctx);
  if (!v) throw new Error("useNav must be used inside <NavProvider>");
  return v;
}
/* ===== 04 文件: src/state/useValenceLoop.tsx 共 103 行 ===== */
import {
  createContext,
  useCallback,
  useContext,
  useEffect,
  useMemo,
  useRef,
  useState,
  type ReactNode,
} from "react";
import { valenceSnapshots, type ValenceSnapshot } from "../data/mock";
type LoopState = {
  current: ValenceSnapshot;
  history: ValenceSnapshot[];
  isLive: boolean;
  loading: boolean;
  lastAiReply: string | null;
  pause: () => void;
  resume: () => void;
  requestAiFeedback: () => Promise<string>;
};
const Ctx = createContext<LoopState | null>(null);
const TICK_MS = 9000;
const HISTORY_LIMIT = 8;
function nowHHMM(): string {
  const date = new Date();
  return `${String(date.getHours()).padStart(2, "0")}:${String(date.getMinutes()).padStart(2, "0")}`;
}
function jitter(snapshot: ValenceSnapshot, salt: number): ValenceSnapshot {
  const delta = ((salt * 53) % 7) - 3;
  const score = Math.max(0, Math.min(100, snapshot.score + delta));
  return { ...snapshot, score, time: nowHHMM() };
}
export function ValenceLoopProvider({ children }: { children: ReactNode }) {

```

```

const [index, setIndex] = useState(0);
const [tick, setTick] = useState(0);
const [isLive, setIsLive] = useState(true);
const [loading, setLoading] = useState(false);
const [lastAiReply, setLastAiReply] = useState<string | null>(null);
const historyRef = useRef<ValenceSnapshot[]>([]);
const [history, setHistory] = useState<ValenceSnapshot[]>([]);
useEffect(() => {
  if (!isLive) return;
  const timer = window.setInterval(() => {
    setTick((value) => value + 1);
    setIndex((value) => (value + 1) % valenceSnapshots.length);
  }, TICK_MS);
  return () => window.clearInterval(timer);
}, [isLive]);
const current = useMemo(() => jitter(valenceSnapshots[index], tick), [index, tick]);
useEffect(() => {
  historyRef.current = [current, ...historyRef.current].slice(0, HISTORY_LIMIT);
  setHistory(historyRef.current);
}, [current]);
const pause = useCallback(() => setIsLive(false), []);
const resume = useCallback(() => setIsLive(true), []);
const requestAiFeedback = useCallback(async () => {
  setLoading(true);
  try {
    const response = await fetch("/api/chat", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        messages: [
          {
            role: "system",
            content: `[Valence 上下文] 等级=${current.level}; score=${current.score}/100; 置信度=${current.confidence}%; 信号质量=${current.signalQuality}%; EEG 摘要=${current.eegSummary}`,
          },
          {
            role: "user",
            content: "请基于以上 Valence 上下文, 用 2 到 3 句中文给我一段温和反馈与一个具体可执行的下一步建议, 不要做医学诊断。",
          },
        ],
      }),
    });
    const data: { reply?: string; error?: string } = await response.json();
    const reply = data.reply?.trim();
    if (!response.ok || !reply) {
      throw new Error(data.error || "AI 反馈暂时不可用。");
    }
    setLastAiReply(reply);
    return reply;
  } catch (error) {
    const fallback = "AI 反馈暂时无法连接。你可以先做一次深呼吸, 等会儿再试。";
    setLastAiReply(fallback);
    throw error instanceof Error ? error : new Error(fallback);
  } finally {
    setLoading(false);
  }
}

```

```

    }
  }, [current]);
  const value: LoopState = useMemo(
    () => ({ current, history, isLive, loading, lastAiReply, pause, resume, requestAiFeedback }),
    [current, history, isLive, loading, lastAiReply, pause, resume, requestAiFeedback],
  );
  return <Ctx.Provider value={value}>{children}</Ctx.Provider>;
}

export function useValenceLoop(): LoopState {
  const value = useContext(Ctx);
  if (!value) throw new Error("useValenceLoop must be used inside <ValenceLoopProvider>");
  return value;
}

/* ===== 05 文件: src/game-session.tsx 共 48 行 ===== */
/* eslint-disable react-refresh/only-export-components */
import { createContext, useContext, useMemo, useState, type ReactNode } from "react";
import { dailyTask, game } from "../data/mock";
export type RaceRoundResult = {
  distance: number;
  averageFocus: number;
  peakFocus: number;
  isNewBest: boolean;
};
export type RaceSessionState = {
  lastAverageFocus: number | null;
  bestDistance: number;
  dailyDoneMinutes: number;
};
type RaceSessionContextValue = RaceSessionState & {
  recordRound: (result: Omit<RaceRoundResult, "isNewBest">) => RaceRoundResult;
};
const INITIAL_SESSION_STATE: RaceSessionState = {
  lastAverageFocus: null,
  bestDistance: 412,
  dailyDoneMinutes: dailyTask.doneMinutes,
};
const RaceSessionContext = createContext<RaceSessionContextValue | null>(null);
export function RaceSessionProvider({ children }: { children: ReactNode }) {
  const [state, setState] = useState<RaceSessionState>(INITIAL_SESSION_STATE);
  const value = useMemo<RaceSessionContextValue>(() => ({
    ...state,
    recordRound: (round) => {
      const isNewBest = round.distance > state.bestDistance;
      const result: RaceRoundResult = { ...round, isNewBest };
      setState((current) => ({
        lastAverageFocus: round.averageFocus,
        bestDistance: Math.max(current.bestDistance, round.distance),
        dailyDoneMinutes: Math.min(dailyTask.totalMinutes, current.dailyDoneMinutes + 0.5),
      }));
      return result;
    },
  })), [state]);
  return <RaceSessionContext.Provider value={value}>{children}</RaceSessionContext.Provider>;
}

export function useRaceSession() {

```

```
    const context = useContext(RaceSessionContext);
    if (!context) throw new Error("useRaceSession must be used inside <RaceSessionProvider>");
    return context;
}
export function getCurrentFocusValue(lastAverageFocus: number | null) {
    return lastAverageFocus ?? game.currentFocus;
}
/* ===== 06 文件: src/data/mock.ts 共 300 行 ===== */
// ===== 智愈莘莘 NeuroHeal • mock 数据（无后端，纯演示） =====
export const user = {
    name: "测试用户",
    level: 4,
    levelName: "守护者",
};
export const device = {
    name: "NeuroBand 头环",
    connected: true,
    battery: 82,
};
export type ValenceLevel = "高效价" | "较高效价" | "较低效价" | "低效价";
export type ValenceSnapshot = {
    id: string;
    time: string;
    score: number;
    level: ValenceLevel;
    confidence: number;
    signalQuality: number;
    eegSummary: string;
    aiFeedback: string;
    recommendedActionId: string;
};
export type InterventionAction = {
    id: string;
    title: string;
    type: "呼吸训练" | "数字处方" | "调节游戏" | "AI陪伴" | "成长记录";
    duration: string;
    target: string;
    reason: string;
    route: SubName;
    params?: Record<string, string>;
};
export type ClosedLoopRecord = {
    id: string;
    startTime: string;
    beforeLevel: ValenceLevel;
    beforeScore: number;
    action: string;
    aiMessage: string;
    afterLevel: ValenceLevel;
    afterScore: number;
    delta: number;
    status: "已完成" | "进行中";
};
export const interventionActions: InterventionAction[] = [
    {
```

```
    id: "breath-alpha",
    title: "3 分钟 Alpha 呼吸调节",
    type: "呼吸训练",
    duration: "3 分钟",
    target: "稳定较低效价状态",
    reason: "当前 EEG 显示 Alpha 波回升空间较大, 适合先用短时呼吸降低紧张感。",
    route: "player",
    params: { from: "Valence 闭环呼吸调节" },
  },
  {
    id: "race-focus",
    title: "意念赛车专注调节",
    type: "调节游戏",
    duration: "30 秒",
    target: "用轻量互动恢复专注",
    reason: "当效价偏低且注意力下降时, 短时游戏可作为非压迫式干预入口。",
    route: "race",
  },
  {
    id: "ai-care",
    title: "与小愈进行 AI 陪伴",
    type: "AI 陪伴",
    duration: "即时",
    target: "获得安抚与下一步建议",
    reason: "AI 根据当前 Valence 等级生成温和反馈, 帮助用户把感受说清楚。",
    route: "aichat",
  },
  {
    id: "prescription-focus",
    title: "考研冲刺专注流",
    type: "数字处方",
    duration: "15 分钟",
    target: "延长稳定专注窗口",
    reason: "当前状态已回升, 可进入结构化专注训练巩固效果。",
    route: "prescription",
    params: { id: "p1" },
  },
];
export const valenceSnapshots: ValenceSnapshot[] = [
  {
    id: "v1",
    time: "09:20",
    score: 42,
    level: "较低效价",
    confidence: 88,
    signalQuality: 92,
    eegSummary: "Alpha 波偏弱, Beta 波短时升高, 提示用户可能存在轻度紧张。",
    aiFeedback: "我注意到你现在有一点紧绷。先不用急着解决所有事, 我们可以从一次短呼吸开始, 让身体先稳下来。",
    recommendedActionId: "breath-alpha",
  },
  {
    id: "v2",
    time: "09:26",
    score: 57,
    level: "较高效价",
```

```
confidence: 90,
signalQuality: 94,
eegSummary: "Alpha 波占比回升, Beta 波波动降低, 状态较上一轮更稳定。",
aiFeedback: "刚才的调节已经让状态回升了一些。你可以保持这个节奏, 继续做一个轻量专注任务。",
recommendedActionId: "prescription-focus",
},
{
  id: "v3",
  time: "09:32",
  score: 64,
  level: "较高效价",
  confidence: 91,
  signalQuality: 95,
  eegSummary: "脑波节律趋于平稳, 情绪效价维持在较积极区间。",
  aiFeedback: "现在的状态比较稳定, 适合进入学习或记录今天的训练感受。",
  recommendedActionId: "prescription-focus",
},
];
export const loopSteps = [
  "EEG 采集",
  "Valence 识别",
  "AI 反馈",
  "干预推荐",
  "再监测",
];
export const closedLoopRecords: ClosedLoopRecord[] = [
  {
    id: "loop-1",
    startTime: "09:20",
    beforeLevel: "较低效价",
    beforeScore: 42,
    action: "3 分钟 Alpha 呼吸调节",
    aiMessage: "先把呼吸放慢, 让身体从紧绷里退出来一点。",
    afterLevel: "较高效价",
    afterScore: 57,
    delta: 15,
    status: "已完成",
  },
  {
    id: "loop-2",
    startTime: "09:32",
    beforeLevel: "较高效价",
    beforeScore: 64,
    action: "考研冲刺专注流",
    aiMessage: "当前状态适合进入短时专注训练, 保持稳定节奏即可。",
    afterLevel: "较高效价",
    afterScore: 67,
    delta: 3,
    status: "进行中",
  },
];
// 首页迷你 EEG 实时波形 (最近若干采样点)
export const eegLive = Array.from({ length: 28 }, (_, i) => ({
  t: i,
  v: 50 + Math.round(28 * Math.sin(i / 1.7) + 8 * Math.sin(i / 0.6)),
}));
```



```

));
// ===== 调节小游戏 (被 game-session 共享, 作为干预入口数据) =====
export const game = {
  title: "意念赛车",
  subtitle: "调节小游戏 • 用专注度驱动",
  currentFocus: 86,
  bestTime: "4' 12" ",
  rules: ["保持专注, 赛车自动加速", "分心时赛车减速, 系统记录状态变化", "完成后回到 EEG 再监测, 更新 Valence 结果"],
};
export const dailyTask = {
  title: "完成 10 分钟专注训练",
  doneMinutes: 6,
  totalMinutes: 10,
  reward: 50,
};
// 底部导航
export type TabKey = "home" | "train" | "community";
export const TABS: { key: TabKey; label: string }[] = [
  { key: "home", label: "监测" },
  { key: "train", label: "干预" },
  { key: "community", label: "记录" },
];
/* ===== 子页面: 设备配对与脑电校准 ===== */
export const pairedDevice = {
  name: "NeuroBand 头环",
  model: "NH-Band 2 • 基础版",
  battery: 82,
  sampleRate: "256 Hz",
  firmware: "v2.4.1",
  connected: true,
  sinceMin: 47,
};
export const otherDevices = [
  { name: "NeuroHeal VR 一体机", note: "未连接", emoji: " " },
  { name: "NeuroHeal 助眠眼罩", note: "未连接", emoji: " " },
];
export const electrodes = [
  { name: "Fp1 • 左前额", quality: 92 },
  { name: "Fp2 • 右前额", quality: 88 },
  { name: "TP9 • 左耳后", quality: 76 },
  { name: "TP10 • 右耳后", quality: 81 },
];
export const calibrationSteps = ["检测电极接触", "采集静息基线", "校准情绪模型", "完成"];
/* ===== 子页面: 数字处方包 (4 大核心课程) ===== */
export type Prescription = {
  id: string; title: string; subtitle: string; tag: string; tone: string; emoji: string;
  band: string; minutes: number; sessions: number; forWho: string;
  desc: string; chapters: { name: string; min: number; done?: boolean }[];
};
export const prescriptions: Prescription[] = [
  {
    id: "pl", title: "考研冲刺专注流", subtitle: "Beta 波强化 • 长时段自习", tag: "专注", tone: "tint-blue", emoji: " ",
    band: "Beta 13 - 30Hz", minutes: 15, sessions: 12, forWho: "考研党 / 备考期大学生",
    desc: "针对备考期注意力涣散、易走神, 用 Beta 波节律引导 + 番茄工作法, 逐步拉长你的「心流窗口」。",
  }
];

```

```

    chapters: [
      { name: "01 • 进入专注：3 分钟脑波热身", min: 3, done: true },
      { name: "02 • 心流维持：12 分钟节律引导", min: 12 },
      { name: "03 • 走神唤醒训练", min: 8 },
      { name: "04 • 收尾复盘：今日专注画像", min: 4 },
    ],
  },
  {
    id: "p2", title: "失眠改善安睡曲", subtitle: "Theta / Delta 引导 • 睡前 30 分钟", tag: "助眠", tone: "tint-teal", emoji: "😴",
    band: "Theta 4-8Hz / Delta", minutes: 20, sessions: 8, forWho: "失眠群体 / 宿舍噪音干扰",
    desc: "睡前用渐慢的 Theta→Delta 频段引导 + 粉噪音，把交感神经「降档」，配合呼吸节律帮助入睡。",
    chapters: [
      { name: "01 • 身体扫描放松", min: 8 },
      { name: "02 • Theta 渐入引导", min: 10 },
      { name: "03 • Delta 深睡铺垫 + 粉噪音", min: 12 },
    ],
  },
  {
    id: "p3", title: "情感创伤疗愈包", subtitle: "正念 + 情绪复盘 • 温和陪伴", tag: "疗愈", tone: "tint-purple", emoji: "💜",
    band: "Alpha 8-13Hz", minutes: 18, sessions: 10, forWho: "经历分手 / 失落 / 低谷期",
    desc: "用 Alpha 波放松配合书写式情绪复盘与自我关怀练习，温和地「看见、接纳、放下」，不评判、不催促。",
    chapters: [
      { name: "01 • 安全着陆：此刻的呼吸", min: 4 },
      { name: "02 • 命名情绪：它在身体哪里", min: 6 },
      { name: "03 • 给过去的自己写一封信", min: 8 },
      { name: "04 • 自我关怀冥想", min: 8 },
    ],
  },
  {
    id: "p4", title: "社交自信提升课", subtitle: "认知重构 + 暴露练习 • 缓解社恐", tag: "成长", tone: "tint-amber", emoji: "🧡",
    band: "Alpha / Beta 混合", minutes: 12, sessions: 6, forWho: "社交焦虑 / 怯场 / 内耗型",
    desc: "针对社交场合心跳加速、脑子空白，用脑波放松 + 微暴露想象 + 认知重构，把「灾难化预期」一点点拆掉。",
    chapters: [
      { name: "01 • 社交前的 90 秒平复", min: 3 },
      { name: "02 • 想象暴露：一次小对话", min: 5 },
      { name: "03 • 拆解「他会怎么看我」", min: 6 },
      { name: "04 • 自信锚点冥想", min: 4 },
    ],
  },
],
/* ===== 子页面：呼吸 / 冥想引导播放器 ===== */
export const breathPlayer = {
  title: "Alpha 波放松训练",
  subtitle: "4-7-8 呼吸法 • 引导音轨",
  totalSec: 300,
  // 一个 4-7-8 周期（秒）
  cycle: { inhale: 4, hold: 7, exhale: 8 },
};
/* ===== 子页面：AI 心理陪伴对话 ===== */
export const aiQuickAsks = ["为什么是较低效价", "先做什么调节", "我想慢慢说一下", "帮我进入呼吸训练"];
export const aiOpening =
  "你好，我是「小愈」。系统刚刚识别到你的 Valence 状态有些波动，我们可以先从一句话说起，也可以直接进入一次短时刻调节。";

```

```

/* ===== 子页面：成长档案（打卡 / 勋章 / 周报） ===== */
// 本月（示例 30 天）：哪些日子已打卡
export const checkinDays = [1, 2, 3, 5, 6, 7, 8, 9, 11, 12, 13, 14, 15, 16, 17, 18, 20, 21, 22, 23, 24, 25, 26, 27];
export const checkinInfo = { monthLabel: "5 月", totalDays: 30, today: 28, streak: 7, monthDone: 24 }
;
export const badges = [
  { name: "初探内心", desc: "完成首次脑电监测", emoji: " ", got: true },
  { name: "7 日专注", desc: "连续打卡 7 天", emoji: " ", got: true },
  { name: "闭环新手", desc: "完成首次「干预前后」闭环记录", emoji: " ", got: true },
  { name: "晨型选手", desc: "连续 5 天 7 点前打卡", emoji: " ", got: true },
  { name: "心流大师", desc: "单次专注 > 30 分钟 ×10", emoji: " ", got: false },
  { name: "安睡达人", desc: "睡眠评分 ≥ 85 连续 7 天", emoji: " ", got: false },
  { name: "处方全勤", desc: "完整跟完一套数字处方", emoji: " ", got: false },
  { name: "破冰勇士", desc: "完成社交自信全部练习", emoji: " ", got: false },
];
export const weeklyReport = {
  range: "5/6 - 5/12",
  mood: [68, 74, 70, 55, 62, 72, 78],
  focus: [70, 66, 74, 80, 78, 84, 86],
  stress: [48, 40, 44, 36, 38, 34, 32],
  days: ["一", "二", "三", "四", "五", "六", "日"],
  risk: "低" as "低" | "中" | "高",
  summary:
    "本周情绪整体平稳偏积极，专注度持续上行；周三下午出现一次明显压力峰值（与课程相关），已通过呼吸训练回落。建议保持睡前正念习惯";
};
/* ===== 子页面路由名 ===== */
export type SubName =
  | "device"
  | "prescription"
  | "player"
  | "aichat"
  | "growth"
  | "race";
/* ===== 07 文件：src/screens/HomeScreen.tsx 共 184 行 ===== */
import { useMemo } from "react";
import { Area, AreaChart, ResponsiveContainer } from "recharts";
import {
  ChevronRight,
  Loader2,
  MessageCircleHeart,
  Radio,
  ShieldCheck,
  Sparkles,
  Waves,
} from "lucide-react";
import { Chip, GaugeRing, Reveal, SectionHeader, Track, useCountUp } from "../components/ui";
import { useNav } from "../nav";
import { useValenceLoop } from "../state/useValenceLoop";
import {
  device,
  eegLive,
  interventionActions,
  loopSteps,
  type ValenceLevel,
} from "../data/mock";

```

```

const LEVEL_TO_STEP: Record<ValenceLevel, number> = {
  低效价: 0,
  较低效价: 1,
  较高效价: 2,
  高效价: 3,
};

function LoopFlow({ activeStep }: { activeStep: number }) {
  return (
    <div className="loop-flow">
      {loopSteps.map((step, index) => (
        <span key={step} className={index === activeStep ? "active" : ""}>
          {step}
        </span>
      ))}
    </div>
  );
}

function eegForLevel(level: ValenceLevel) {
  const amplitude = level === "低效价" ? 36 : level === "较低效价" ? 28 : level === "较高效价" ? 18 : 12;
  return eegLive.map((point) => ({
    t: point.t,
    v: 50 + Math.round(amplitude * Math.sin(point.t / 1.7) + (amplitude / 3) * Math.sin(point.t / 0.6)),
  }));
}

export default function HomeScreen() {
  const { openSub } = useNav();
  const { current, loading, lastAiReply, requestAiFeedback } = useValenceLoop();
  const score = useCountUp(current.score, 700, [current.id, current.score]);
  const intervention = useMemo(
    () => interventionActions.find((item) => item.id === current.recommendedActionId) ?? interventionActions[0],
    [current.recommendedActionId],
  );
  const eegData = useMemo(() => eegForLevel(current.level), [current.level]);
  const activeStep = loading ? 2 : LEVEL_TO_STEP[current.level];
  const handleAiFeedback = () => {
    void requestAiFeedback().catch(() => undefined);
  };
  return (
    <div className="screen">
      <Reveal i={0}>
        <div className="row between">
          <div className="col" style={{ gap: 4 }}>
            <span className="kicker">实时闭环监测</span>
            <span className="h1">EEG-Valence 控制台</span>
          </div>
          <Chip variant="teal"><Radio size={13} /> 实时</Chip>
        </div>
      </Reveal>
      <Reveal i={1}>
        <div className="card monitor-hero">
          <div className="row between top">
            <div className="col" style={{ gap: 4 }}>
              <span className="tiny">当前情绪效价</span>
            </div>
          </div>
        </div>
      </Reveal>
    </div>
  );
}

```

```

    <span className="monitor-level">{current.level}</span>
    <span className="muted">Valence 分数 {current.score}/100 • {current.time}</span>
  </div>
  <GaugeRing
    value={current.score}
    size={118}
    stroke={11}
    trackColor="#dcecf2"
    from="var(--teal)"
    to="var(--brand)"
    glow={false}
  >
    <span className="num" style={{ fontSize: 28, fontWeight: 700, color: "var(--brand-deep
)" }}>{score}</span>
    <span className="tiny">Valence</span>
  </GaugeRing>
</div>
<div className="monitor-signal-grid">
  <div>
    <span>模型置信度</span>
    <strong>{current.confidence}%</strong>
    <Track pct={current.confidence} color="var(--brand)" />
  </div>
  <div>
    <span>信号质量</span>
    <strong>{current.signalQuality}%</strong>
    <Track pct={current.signalQuality} color="var(--teal)" />
  </div>
</div>
<div className="monitor-eeg-panel">
  <div className="row between">
    <span className="row" style={{ gap: 8 }}>
      <Waves size={16} color="var(--brand-deep)" />
      <span className="body" style={{ fontWeight: 700 }}>实时 EEG 波形</span>
    </span>
    <button
      className="chip"
      style={{ background: "var(--blue-soft)", cursor: "pointer" }}
      onClick={() => openSub("device")}
      aria-label="打开设备与校准"
    >
      {device.name} • {device.battery}%
    </button>
  </div>
  <div className="chart-box sm">
    <ResponsiveContainer>
      <AreaChart data={eegData} margin={{ top: 4, right: 0, bottom: 0, left: 0 }}>
        <defs>
          <linearGradient id="homeEegGrad" x1="0" y1="0" x2="0" y2="1">
            <stop offset="0%" stopColor="var(--brand)" stopOpacity={0.28} />
            <stop offset="100%" stopColor="var(--brand)" stopOpacity={0} />
          </linearGradient>
        </defs>
        <Area type="monotone" dataKey="v" stroke="var(--brand-deep)" strokeWidth={2.2} fill=
"url(#homeEegGrad)" isAnimationActive />

```

```

        </AreaChart>
      </ResponsiveContainer>
    </div>
    <span className="tiny">{current.eegSummary}</span>
  </div>
</div>
</Reveal>
<Reveal i={2}>
  <div className="card">
    <SectionHeader title="闭环运行流程" />
    <LoopFlow activeStep={activeStep} />
    <div className="row" style={{ gap: 8 }}>
      <ShieldCheck size={15} color="var(--teal-deep)" />
      <span className="tiny">EEG 采集 → Valence 识别 → AI 反馈 → 干预 → 再监测, 循环刷新。</span>
    </div>
  </div>
</Reveal>
<Reveal i={3}>
  <div className="card ai-feedback-card">
    <div className="row top" style={{ gap: 12 }}>
      <div className="icon-badge" style={{ background: "var(--blue-soft)" }}>
        <MessageCircleHeart size={19} color="var(--brand-deep)" />
      </div>
      <div className="col grow">
        <span className="title">AI 当前反馈</span>
        <span className="body">{lastAiReply ?? current.aiFeedback}</span>
      </div>
    </div>
    <button
      className="btn btn-ghost"
      onClick={handleAiFeedback}
      disabled={loading}
      style={{ alignSelf: "flex-start" }}
    >
      {loading ? <Loader2 size={15} className="spin" /> : <Sparkles size={15} />}
      {loading ? "正在请求 AI 反馈..." : lastAiReply ? "再次请求 AI 反馈" : "请求 AI 反馈"}
    </button>
    <div className="recommended-action">
      <div className="col grow">
        <span className="tiny">推荐干预</span>
        <strong>{intervention.title}</strong>
        <small>{intervention.reason}</small>
      </div>
      <button className="btn btn-primary" onClick={() => openSub(intervention.route, intervent
ion.params)}>
        开始 <ChevronRight size={16} />
      </button>
    </div>
  </div>
</Reveal>
</div>
);
}
/* ===== 08 文件: src/screens/TrainScreen.tsx 共 103 行 ===== */
import { useMemo } from "react";

```

```

import { Activity, ChevronRight, Clock, Flower2, Play, Sparkles, Wind } from "lucide-react";
import { Chip, Reveal, SectionHeader } from "../components/ui";
import { useNav } from "../nav";
import { interventionActions, prescriptions } from "../data/mock";
import { useValenceLoop } from "../state/useValenceLoop";
export default function TrainScreen() {
  const { openSub } = useNav();
  const { current } = useValenceLoop();
  const recommendedIntervention = useMemo(
    () => interventionActions.find((item) => item.id === current.recommendedActionId) ?? interventio
nActions[0],
    [current.recommendedActionId],
  );
  return (
    <div className="screen">
      <Reveal i={0}>
        <div className="row between">
          <div className="col" style={{ gap: 3 }}>
            <span className="kicker">AI 干预中心</span>
            <span className="h1">根据 Valence 推荐行动</span>
          </div>
          <Chip variant="teal"><Sparkles size={13} /> AI 推荐</Chip>
        </div>
      </Reveal>
      <Reveal i={1}>
        <div className="card intervention-hero">
          <div className="row between top">
            <div className="col grow">
              <span className="tiny">当前状态</span>
              <span className="title">{current.level} • Valence {current.score}</span>
              <span className="muted">{current.aiFeedback}</span>
            </div>
            <div className="intervention-score num">{current.score}</div>
          </div>
          <div className="recommended-action highlight">
            <div className="icon-badge" style={{ background: "var(--teal-soft)" }}>
              <Wind size={19} color="var(--teal-deep)" />
            </div>
            <div className="col grow">
              <span className="tiny">优先建议</span>
              <strong>{recommendedIntervention.title}</strong>
              <small>{recommendedIntervention.target} • {recommendedIntervention.duration}</small>
            </div>
            <button className="btn btn-primary" onClick={() => openSub(recommendedIntervention.route
, recommendedIntervention.params)}>
              开始 <ChevronRight size={16} />
            </button>
          </div>
        </div>
      </Reveal>
      <Reveal i={2}>
        <SectionHeader title="可选干预方式" />
      </Reveal>
      {interventionActions.map((action, index) => (
        <Reveal i={3 + index} key={action.id}>

```

```

        <button className="card intervention-action-card" onClick={() => openSub(action.route, action.params)}>
          <div className="row top" style={{ gap: 12 }}>
            <div className="icon-badge" style={{ background: action.id === recommendedIntervention.id ? "var(--teal-soft)" : "var(--blue-soft)" }}>
              {action.type === "调节游戏" ? <Play size={18} color="var(--brand-deep)" /> : <Flower2 size={18} color="var(--teal-deep)" />}
            </div>
            <div className="col grow" style={{ gap: 3 }}>
              <span className="row" style={{ gap: 7, flexWrap: "wrap" }}>
                <span className="body" style={{ fontWeight: 700 }}>{action.title}</span>
                <Chip variant={action.id === recommendedIntervention.id ? "teal" : ""}>{action.type}</Chip>
              </span>
              <span className="muted">{action.reason}</span>
              <span className="tiny"><Clock size={11} style={{ verticalAlign: "-2px" }} /> {action.duration} • {action.target}</span>
            </div>
            <ChevronRight size={16} color="var(--t-tertiary)" />
          </div>
        </button>
      </Reveal>
    ))}
    <Reveal i={7}>
      <SectionHeader title="数字处方包" />
    </Reveal>
    {prescriptions.map((course, idx) => (
      <Reveal i={8 + idx} key={course.id}>
        <button className={`card ${course.emoji}`} style={{ flexDirection: "row", alignItems: "center", gap: 12, width: "100%", textAlign: "left" }} onClick={() => openSub("prescription", { id: course.id })}>
          <div className="icon-badge shadow" style={{ background: "#fff", width: 46, height: 46, fontSize: 20 }}>{course.emoji}</div>
          <div className="col grow" style={{ gap: 3 }}>
            <span className="body" style={{ fontWeight: 700 }}>{course.title}</span>
            <span className="tiny" style={{ color: "var(--t-secondary)" }}>{course.subtitle}</span>
          </div>
          <span className="tiny"><Activity size={10} style={{ verticalAlign: "-1px" }} /> {course.band} • {course.minutes} 分钟</span>
        </div>
        <ChevronRight size={16} color="var(--t-tertiary)" />
      </button>
    </Reveal>
  ))}
</div>
);
}
/* ===== 09 文件: src/screens/CommunityScreen.tsx 共 134 行 ===== */
import {
  Award,
  Bell,
  CalendarCheck2,
  ChevronRight,
  Crown,
  Flame,

```



```

    Trophy,
  } from "lucide-react";
import { Chip, Reveal, Track } from "../components/ui";
import {
  badges,
  checkinInfo,
  closedLoopRecords,
  user,
} from "../data/mock";
import { useNav } from "../nav";
import { useValenceLoop } from "../state/useValenceLoop";
export default function CommunityScreen() {
  const { openSub } = useNav();
  const { current } = useValenceLoop();
  const completedLoops = closedLoopRecords.filter((record) => record.status === "已完成").length;
  const ownedBadges = badges.filter((badge) => badge.got).slice(0, 4);
  const checkinPct = Math.round((checkinInfo.monthDone / checkinInfo.totalDays) * 100);
  return (
    <div className="screen">
      <Reveal i={0}>
        <div className="row between">
          <div className="col" style={{ gap: 3 }}>
            <span className="kicker">闭环记录 • 成长归档</span>
            <span className="h1">个人记录</span>
          </div>
          <button
            className="avатар notice-trigger"
            style={{ width: 38, height: 38, background: "#fff", boxShadow: "var(--shadow-soft)" }}
            aria-label="成长档案"
            onClick={() => openSub("growth")}
          >
            <Bell size={17} color="var(--brand-deep)" />
          </button>
        </div>
      </Reveal>
      <Reveal i={1}>
        <div className="card record-hero">
          <div className="row between" style={{ alignItems: "flex-start" }}>
            <div className="col">
              <span className="tiny">当前 Valence 状态</span>
              <span className="title">{current.level} • Valence {current.score}</span>
              <span className="muted">已完成 {completedLoops} 次闭环反馈 • 连续打卡 {checkinInfo.streak} 天</span>
            </div>
            <Chip variant="teal"><Crown size={12} /> Lv. {user.level} • {user.levelName}</Chip>
          </div>
          <Track pct={checkinPct} color="var(--teal)" />
          <button className="btn btn-primary" onClick={() => openSub("growth")}><CalendarCheck2 size={15} /> 打开成长档案</button>
        </div>
      </Reveal>
      <Reveal i={2}>
        <div className="card">
          <div className="section-head">
            <span className="title">闭环记录时间轴</span>

```

```

    </div>
    <div className="loop-timeline">
      {closedLoopRecords.map((record) => {
        const isPositive = record.delta >= 0;
        return (
          <div className="loop-timeline-row" key={record.id}>
            <div className="loop-timeline-dot" data-status={record.status} />
            <div className="loop-timeline-body">
              <div className="row between">
                <span className="body" style={{ fontWeight: 700 }}>{record.startTime} • {record.action}</span>
                <span className={`loop-delta ${isPositive ? "up" : "down"}`}>
                  {isPositive ? "+" : ""}{record.delta}
                </span>
              </div>
              <div className="row" style={{ gap: 8, flexWrap: "wrap" }}>
                <Chip variant={record.beforeLevel.includes("低") ? "amber" : "teal"}>
                  干预前 {record.beforeLevel} • {record.beforeScore}
                </Chip>
                <ChevronRight size={13} color="var(--t-tertiary)" />
                <Chip variant={record.afterLevel.includes("低") ? "amber" : "teal"}>
                  干预后 {record.afterLevel} • {record.afterScore}
                </Chip>
                <span className="chip" style={{ background: record.status === "已完成" ? "var(--teal-soft)" : "var(--blue-soft)", color: record.status === "已完成" ? "var(--teal-deep)" : "var(--brand-deep)" }}>{record.status}</span>
              </div>
              <span className="muted">{record.aiMessage}</span>
            </div>
          </div>
        );
      })}
    </div>
  </div>
</Reveal>
<Reveal i={3}>
  <div className="card">
    <div className="section-head">
      <span className="title">成长档案 • 本月概览</span>
      <button className="link" onClick={() => openSub("growth")}>查看全部 <ChevronRight size={13} /></button>
    </div>
    <div className="growth-grid">
      <div className="growth-stat">
        <Flame size={16} color="var(--stress)" />
        <strong>{checkinInfo.streak}</strong>
        <span>连续打卡天</span>
      </div>
      <div className="growth-stat">
        <Trophy size={16} color="var(--joy-deep)" />
        <strong>{checkinInfo.monthDone}/{checkinInfo.totalDays}</strong>
        <span>本月打卡</span>
      </div>
      <div className="growth-stat">
        <Award size={16} color="var(--purple-deep)" />

```

```

        <strong>{ownedBadges.length}/{badges.length}</strong>
        <span>已获勋章</span>
      </div>
    </div>
    <div className="badge-strip">
      {ownedBadges.map((badge) => (
        <div className="badge-chip" key={badge.name} title={badge.desc}>
          <span className="badge-emoji">{badge.emoji}</span>
          <span className="tiny">{badge.name}</span>
        </div>
      ))}
    </div>
  </div>
</Reveal>
</div>
);
}
/* ===== 10 文件: src/screens/sub/DeviceScreen.tsx 共 159 行 ===== */
import { useEffect, useState } from "react";
import { motion, AnimatePresence } from "framer-motion";
import { BluetoothConnected, BluetoothSearching, Activity, RefreshCw, ShieldCheck, Plus, CircleCheck }
  from "lucide-react";
import SubScreen from "../../components/SubScreen";
import { Chip, Reveal } from "../../components/ui";
import { pairedDevice, otherDevices, electrodes, calibrationSteps } from "../../data/mock";
function qColor(q: number) {
  return q >= 85 ? "var(--calm)" : q >= 70 ? "var(--joy)" : "var(--stress)";
}
export default function DeviceScreen() {
  // 信号质量轻微跳动
  const [tick, setTick] = useState(0);
  useEffect(() => {
    const id = setInterval(() => setTick((t) => t + 1), 1400);
    return () => clearInterval(id);
  }, []);
  const live = electrodes.map((e, i) => ({
    ...e,
    q: Math.max(58, Math.min(98, e.quality + Math.round(Math.sin(tick + i) * 4))),
  }));
  // 校准流程
  const [calibrating, setCalibrating] = useState(false);
  const [step, setStep] = useState(0);
  useEffect(() => {
    if (!calibrating) return;
    if (step >= calibrationSteps.length - 1) {
      const t = setTimeout(() => setCalibrating(false), 1200);
      return () => clearTimeout(t);
    }
    const t = setTimeout(() => setStep((s) => s + 1), 1100);
    return () => clearTimeout(t);
  }, [calibrating, step]);
  const startCalib = () => { setStep(0); setCalibrating(true); };
  return (
    <SubScreen title="设备与脑电校准">
      <Reveal i={0}>

```

```

    <div className="card hero g-hero">
      <div className="row between">
        <span className="kicker" style={{ color: "rgba(255,255,255,.78)" }}>已连接设备</span>
        <Chip variant="glass live">实时</Chip>
      </div>
      <div className="row" style={{ gap: 14 }}>
        <div className="avatar" style={{ width: 54, height: 54, background: "rgba(255,255,255,.2)" }}>
          <BluetoothConnected size={24} color="#fff" />
        </div>
        <div className="col grow">
          <span className="h2">{pairedDevice.name}</span>
          <span className="muted on-80">{pairedDevice.model} • 已连接 {pairedDevice.sinceMin} 分钟</span>
        </div>
      </div>
      <div className="hero-mini-stats">
        <div><div className="tiny on-70" style={{ fontWeight: 600 }}>电量</div><div className="num" style={{ fontSize: 18, fontWeight: 700, marginTop: 3 }}>{pairedDevice.battery}%</div></div>
        <div><div className="tiny on-70" style={{ fontWeight: 600 }}>采样率</div><div className="num" style={{ fontSize: 18, fontWeight: 700, marginTop: 3 }}>{pairedDevice.sampleRate}</div></div>
        <div><div className="tiny on-70" style={{ fontWeight: 600 }}>固件</div><div className="num" style={{ fontSize: 18, fontWeight: 700, marginTop: 3 }}>{pairedDevice.firmware}</div></div>
      </div>
    </Reveal>
    { /* 电极信号质量 */ }
    <Reveal i={1}>
      <div className="card">
        <div className="row between">
          <span className="row" style={{ gap: 8 }}>
            <span className="icon-badge" style={{ width: 28, height: 28, borderRadius: 9, background: "var(--blue-soft)" }}><Activity size={15} color="var(--brand)" /></span>
            <span className="title">电极信号质量</span>
          </span>
          <Chip variant="teal">4 路 • 干电极</Chip>
        </div>
        {live.map((e) => (
          <div className="row" key={e.name} style={{ gap: 10 }}>
            <span className="muted" style={{ width: 92, fontSize: 11.5 }}>{e.name}</span>
            <div className="track grow"><motion.i animate={{ width: `${e.q}%` }} transition={{ duration: 0.8 }} style={{ background: qColor(e.q) }} /></div>
            <span className="num" style={{ width: 36, textAlign: "right", fontSize: 12, fontWeight: 700, color: qColor(e.q) }}>{e.q}</span>
          </div>
        ))}
        <span className="tiny">信号良好。如某路偏低，请轻压电极并确保与皮肤贴合。</span>
      </div>
    </Reveal>
    { /* 校准 */ }
    <Reveal i={2}>
      <div className="card">
        <span className="title">情绪模型校准</span>
        <span className="muted">采集 90 秒静息基线，让情绪识别更贴合你本人。建议每周或更换佩戴位置后做一次。</span>
        <AnimatePresence mode="wait">

```

```

    {calibrating ? (
      <motion.div key="calib" initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opac
ity: 0 }} className="col" style={{ gap: 12, alignItems: "center", padding: "8px 0" }}>
        <div className="gauge" style={{ width: 96, height: 96 }}>
          <svg viewBox="0 0 96 96">
            <circle cx="48" cy="48" r="41" fill="none" stroke="#e9f1f6" strokeWidth="8" />
            <motion.circle cx="48" cy="48" r="41" fill="none" stroke="var(--brand)" strokeWi
dth="8" strokeLinecap="round"
              strokeDasharray={2 * Math.PI * 41}
              animate={{ strokeDashoffset: 2 * Math.PI * 41 * (1 - (step + 1) / calibrationS
teps.length) }}
              transition={{ duration: 0.9 }} />
          </svg>
          <div className="gauge-center"><span className="num" style={{ fontSize: 22, fontWei
ght: 700, color: "var(--brand-deep)" }}>{Math.round(((step + 1) / calibrationSteps.length) * 100)}%<
/span></div>
        </div>
        <div className="col" style={{ gap: 6, width: "100%" }}>
          {calibrationSteps.map((s, i) => (
            <div className="row" key={s} style={{ gap: 8 }}>
              {i < step || (i === calibrationSteps.length - 1 && step === calibrationSteps.l
ength - 1)
                ? <CircleCheck size={16} color="var(--calm)" />
                : i === step ? <RefreshCw size={16} color="var(--brand)" className="spin" />
                : <span style={{ width: 16, height: 16, borderRadius: 999, border: "2px soli
d var(--hairline)" }} />
              <span className="body" style={{ color: i <= step ? "var(--t-primary)" : "var(-
t-tertiary)" }}>{s}</span>
            </div>
          ))}
        </div>
      </motion.div>
    ) : (
      <motion.button key="btn" initial={{ opacity: 0 }} animate={{ opacity: 1 }} className="
btn btn-primary btn-block" onClick={startCalib}>
        <RefreshCw size={16} /> 开始校准 (约 90 秒)
      </motion.button>
    )}
  </AnimatePresence>
</div>
</Reveal>
{ /* 其他设备 */ }
<Reveal i={3}>
  <div className="card">
    <span className="title">我的设备</span>
    {otherDevices.map((d) => (
      <div className="list-row" key={d.name}>
        <div className="icon-badge" style={{ width: 40, height: 40, background: "var(--blue-so
ft)", fontSize: 18 }}>{d.emoji}</div>
        <div className="col grow"><span className="body" style={{ fontWeight: 600 }}>{d.name}<
/span><span className="tiny">{d.note}</span></div>
        <span className="chip"><BluetoothSearching size={12} /> 连接</span>
      </div>
    ))}
    <button className="btn btn-ghost btn-block"><Plus size={16} /> 添加新设备</button>
  </div>

```

```

    </div>
  </Reveal>
  <Reveal i={4}>
    <div className="card tint-teal" style={{ flexDirection: "row", alignItems: "flex-start", gap
: 10 }}>
      <ShieldCheck size={18} color="var(--teal-deep)" style={{ marginTop: 2, flex: "none" }} />
      <span className="muted">采集到的脑电数据全程端侧加密，仅你本人与你授权的校心理中心可见，绝不对外共享。</span>
    </div>
  </Reveal>
</SubScreen>
);
}
/* ===== 11 文件: src/screens/sub/PrescriptionScreen.tsx 共 75 行 ===== */
import { Play, CircleCheck, Activity, Clock, Sparkles, Headphones } from "lucide-react";
import SubScreen from "../../components/SubScreen";
import { Chip, Reveal } from "../../components/ui";
import { useNav } from "../../nav";
import { prescriptions } from "../../data/mock";
export default function PrescriptionScreen({ id }: { id?: string }) {
  const { openSub } = useNav();
  const p = prescriptions.find((x) => x.id === id) ?? prescriptions[0];
  const done = p.chapters.filter((c) => c.done).length;
  return (
    <SubScreen title="个性化数字处方">
      <Reveal i={0}>
        <div className={`card ${p.tone}`} style={{ gap: 14 }}>
          <div className="row" style={{ gap: 14 }}>
            <div className="icon-badge shadow" style={{ width: 56, height: 56, background: "#fff", f
ontSize: 28 }}>{p.emoji}</div>
            <div className="col grow">
              <span className="h1">{p.title}</span>
              <span className="muted">{p.subtitle}</span>
            </div>
          </div>
          <div className="pill-row">
            <Chip variant="solid-blue">{p.tag}</Chip>
            <Chip><Activity size={12} /> {p.band}</Chip>
            <Chip><Clock size={12} /> {p.minutes} 分钟/次</Chip>
            <Chip><Headphones size={12} /> {p.sessions} 节</Chip>
          </div>
        </div>
      </Reveal>
      <Reveal i={1}>
        <div className="card">
          <span className="title">这套处方为谁而设</span>
          <span className="body" style={{ color: "var(--brand-deep)", fontWeight: 600 }}>{p.forWho}<
/span>
          <span className="muted">{p.desc}</span>
        </div>
      </Reveal>
      <Reveal i={2}>
        <div className="card">
          <div className="section-head">
            <span className="title">课程章节</span>
            <span className="tiny">{done} / {p.chapters.length} 已完成</span>
          </div>
        </div>
      </Reveal>
    </SubScreen>
  );
}

```

```

    </div>
    {p.chapters.map((c, idx) => (
      <div className="list-row" key={c.name}>
        {c.done
          ? <CircleCheck size={20} color="var(--calm)" style={{ flex: "none" }} />
          : <div className="icon-badge" style={{ width: 26, height: 26, borderRadius: 999, bac
kground: "var(--blue-soft)", fontSize: 11, fontWeight: 700, color: "var(--brand-deep)" }}>{idx + 1}<
/div>}
        <div className="col grow"><span className="body" style={{ fontWeight: 600 }}>{c.name}<
/span><span className="tiny">{c.min} 分钟</span></div>
        <button className="chip" onClick={() => openSub("player", { from: p.title, chapter: c.
name })}><Play size={11} /> 播放</button>
      </div>
    ))}
  </div>
</Reveal>
<Reveal i={3}>
  <div className="card tint-amber" style={{ flexDirection: "row", alignItems: "center", gap: 1
0 }}>
    <Sparkles size={16} color="var(--joy-deep)" style={{ flex: "none" }} />
    <span className="muted">完整跟完本套处方将解锁 <b style={{ color: "var(--joy-deep)" }}>「处方全勤」勋章</b>, 并
自动写入成长档案。</span>
  </div>
</Reveal>
<Reveal i={4}>
  <button className="btn btn-primary btn-block" onClick={() => openSub("player", { from: p.tit
le })}>
    <Play size={16} /> 开始今日训练
  </button>
</Reveal>
</SubScreen>
);
}
/* ===== 12 文件: src/screens/sub/PlayerScreen.tsx 共 123 行 ===== */
import { useEffect, useRef, useState } from "react";
import { motion, AnimatePresence } from "framer-motion";
import { Pause, Play, X, RotateCw, Volume2 } from "lucide-react";
import { useNav } from "../../nav";
import { breathPlayer } from "../../data/mock";
type Phase = "inhale" | "hold" | "exhale";
const LABEL: Record<Phase, string> = { inhale: "吸气", hold: "屏息", exhale: "呼气" };
const SEQ: [Phase, number][] = [["inhale", 4000], ["hold", 7000], ["exhale", 8000]];
function fmt(s: number) {
  const m = Math.floor(s / 60);
  const sec = Math.floor(s % 60);
  return `${m}:${String(sec).padStart(2, "0")}`;
}
export default function PlayerScreen({ from }: { from?: string }) {
  const { closeSub } = useNav();
  const total = breathPlayer.totalSec;
  const [elapsed, setElapsed] = useState(0);
  const [phase, setPhase] = useState<Phase>("inhale");
  const [playing, setPlaying] = useState(true);
  const [done, setDone] = useState(false);
  const phaseIdx = useRef(0);

```

```

// progress (accelerated for demo: ~22s to finish)
useEffect(() => {
  if (!playing || done) return;
  const id = setInterval(() => {
    setElapsed((e) => {
      const n = e + total * 0.011;
      if (n >= total) { setDone(true); return total; }
      return n;
    });
  }, 240);
  return () => clearInterval(id);
}, [playing, done, total]);
// breathing phase cycle (real 4-7-8 timing)
useEffect(() => {
  if (!playing || done) return;
  let timer: number;
  const run = () => {
    setPhase(SEQ[phaseIdx.current][0]);
    timer = window.setTimeout(() => {
      phaseIdx.current = (phaseIdx.current + 1) % SEQ.length;
      run();
    }, SEQ[phaseIdx.current][1]);
  };
  run();
  return () => clearTimeout(timer);
}, [playing, done]);
const relax = Math.round(62 + 26 * (elapsed / total));
const scale = phase === "inhale" ? 1.18 : phase === "exhale" ? 0.72 : 1.18;
const dur = phase === "inhale" ? 4 : phase === "exhale" ? 8 : 0.2;
const restart = () => { setElapsed(0); setDone(false); setPlaying(true); phaseIdx.current = 0; };
return (
  <div className="sub-screen player-screen">
    <div className="sub-header" style={{ background: "transparent", borderBottom: "none" }}>
      <button className="sub-back" onClick={closeSub} aria-label="关闭"><X size={20} color="var(--t-
primary)" /></button>
      <span className="sub-title">{from ?? breathPlayer.title}</span>
      <div className="sub-head-right"><Volume2 size={18} color="var(--t-tertiary)" /></div>
    </div>
    <div className="player-body">
      <span className="kicker" style={{ textAlign: "center" }}>{breathPlayer.subtitle}</span>
      <div className="breath-wrap">
        <motion.span className="breath-ring r1" animate={{ scale, opacity: phase === "exhale" ? 0.
3 : 0.55 }} transition={{ duration: dur, ease: "easeInOut" }} />
        <motion.span className="breath-ring r2" animate={{ scale: scale * 0.86 }} transition={{ du
ration: dur, ease: "easeInOut" }} />
        <motion.div className="breath-core" animate={{ scale: scale * 0.72 }} transition={{ durati
on: dur, ease: "easeInOut" }}>
          <span className="breath-label">{LABEL[phase]}</span>
        </motion.div>
      </div>
    </div>
    <div className="player-stats">
      <div className="col" style={{ alignItems: "center", gap: 2 }}>
        <span className="tiny">已练习</span>
        <span className="num metric-sm" style={{ color: "var(--brand-deep)" }}>{fmt(elapsed)}</s
pan>

```



```

    </div>
    <span className="divider-v" />
    <div className="col" style={{ alignItems: "center", gap: 2 }}>
      <span className="tiny">实时放松度</span>
      <span className="num metric-sm" style={{ color: "var(--teal-deep)" }}>{relax}%</span>
    </div>
    <span className="divider-v" />
    <div className="col" style={{ alignItems: "center", gap: 2 }}>
      <span className="tiny">总时长</span>
      <span className="num metric-sm" style={{ color: "var(--t-secondary)" }}>{fmt(total)}</span>
    </div>
  </div>
</div>
<div className="track" style={{ width: "100%", maxWidth: 300 }}>
  <i style={{ width: `${(elapsed / total) * 100}%`, background: "var(--grad-btn)" }} />
</div>
<div className="row" style={{ gap: 16, marginTop: 4 }}>
  <button className="player-btn ghost" onClick={restart}><RotateCw size={18} /></button>
  <button className="player-btn main" onClick={() => setPlaying((p) => !p)}>{playing ? <Pause size={24} fill="#fff" /> : <Play size={24} fill="#fff" />}</button>
  <button className="player-btn ghost" onClick={() => setDone(true)}><span style={{ fontSize: 11, fontWeight: 700 }}>结束</span></button>
</div>
<span className="tiny" style={{ textAlign: "center", maxWidth: 260 }}>跟随光圈：慢慢吸气 4 秒、屏息 7 秒、缓缓呼气 8 秒。把注意力放在呼吸上就好。</span>
</div>
<AnimatePresence>
  {done && (
    <motion.div className="player-done-mask" initial={{ opacity: 0 }} animate={{ opacity: 1 }}
    exit={{ opacity: 0 }}>
      <motion.div className="card player-done" initial={{ scale: 0.9, y: 16 }} animate={{ scale: 1, y: 0 }} transition={{ type: "spring", stiffness: 280, damping: 22 }}>
        <div style={{ fontSize: 40 }}></div>
        <span className="hl">训练完成！</span>
        <span className="muted" style={{ textAlign: "center" }}>放松度提升 <b style={{ color: "var(--teal-deep)" }}><↑ 26</b>，副交感神经更活跃了。本次训练已写入成长档案。</span>
        <div className="row" style={{ gap: 10, width: "100%" }}>
          <button className="btn btn-ghost grow" onClick={restart}>再来一组</button>
          <button className="btn btn-primary grow" onClick={closeSub}>完成</button>
        </div>
      </motion.div>
    </motion.div>
  )}
</AnimatePresence>
</div>
);
}
/* ===== 13 文件：src/screens/sub/AiChatScreen.tsx 共 200 行 ===== */
import { useEffect, useRef, useState } from "react";
import { AnimatePresence, motion } from "framer-motion";
import { CalendarCheck, PhoneCall, Send, ShieldCheck, Sparkle, Volume2, VolumeX } from "lucide-react";
import SubScreen from "../../components/SubScreen";
import { aiOpening, aiQuickAsks, interventionActions } from "../../data/mock";
import { useSpeechGuide } from "../../hooks/useSpeechGuide";

```

```

import { useValenceLoop } from "../../state/useValenceLoop";
type Msg = { id: number; role: "ai" | "me"; text: string };
type ApiMessage = { role: "system" | "assistant" | "user"; content: string };
type ChatResponse = { reply?: string; error?: string };
const CHAT_HISTORY_LIMIT = 12;
const NETWORK_ERROR_REPLY = "我刚刚和服务端连线时遇到一点问题。你可以稍后再试一次，或者先把想说的话留在这里。";
function toApiMessages(messages: Msg[]): ApiMessage[] {
  return messages.slice(-CHAT_HISTORY_LIMIT).map((message) => ({
    role: message.role === "ai" ? "assistant" : "user",
    content: message.text,
  }));
}
export default function AiChatScreen() {
  const [msgs, setMsgs] = useState<Msg[]>([ { id: 0, role: "ai", text: aiOpening } ]);
  const [input, setInput] = useState("");
  const [typing, setTyping] = useState(false);
  const [toast, setToast] = useState<string | null>(null);
  const idRef = useRef(1);
  const endRef = useRef<HTMLDivElement>(null);
  const introSpokenRef = useRef(false);
  const speech = useSpeechGuide(true);
  const { current } = useValenceLoop();
  const currentIntervention = interventionActions.find((item) => item.id === current.recommendedActi
    onId) ?? interventionActions[0];
  useEffect(() => {
    endRef.current?.scrollIntoView({ behavior: "smooth" });
  }, [msgs, typing]);
  useEffect(() => {
    if (introSpokenRef.current || !speech.enabled) return;
    introSpokenRef.current = true;
    const timer = window.setTimeout(() => {
      speech.speak(`你已进入 AI 心理陪伴界面。${aiOpening}`);
    }, 450);
    return () => window.clearTimeout(timer);
  }, [speech, speech.enabled]);
  const send = async (text: string) => {
    const content = text.trim();
    if (!content || typing) return;
    const nextMessage = { id: idRef.current++, role: "me" as const, text: content };
    const nextMsgs = [...msgs, nextMessage];
    setMsgs(nextMsgs);
    setInput("");
    setTyping(true);
    const valenceContext: ApiMessage = {
      role: "system",
      content: `[Valence 上下文] 等级=${current.level}; score=${current.score}/100; 置信度=${current.confidenc
        e}%; 信号质量=${current.signalQuality}%; EEG 摘要=${current.eegSummary}`;
    };
    try {
      const response = await fetch("/api/chat", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ messages: [valenceContext, ...toApiMessages(nextMsgs)] }),
      });
      const data = await response.json() as ChatResponse;

```

```

    const reply = data.reply?.trim();
    if (!response.ok || !reply) {
      throw new Error(data.error || "DeepSeek response is empty.");
    }
    setMsgs((messages) => [...messages, { id: idRef.current++, role: "ai", text: reply }]);
    speech.speak(reply);
  } catch (error) {
    console.error(error);
    setMsgs((messages) => [...messages, { id: idRef.current++, role: "ai", text: NETWORK_ERROR_REPLY }]);
    speech.speak(NETWORK_ERROR_REPLY);
  } finally {
    setTyping(false);
  }
};

const fireToast = (message: string) => {
  setToast(message);
  speech.speak(message);
  setTimeout(() => setToast(null), 2600);
};

const toggleSpeech = () => {
  if (!speech.supported) {
    fireToast("当前浏览器暂不支持语音播报");
    return;
  }
  const nextEnabled = !speech.enabled;
  speech.toggle();
  fireToast(nextEnabled ? "语音陪伴已开启，小愈会读出回复" : "语音陪伴已关闭");
};

return (
  <SubScreen
    title="AI 心理陪伴 • 小愈"
    bodyClassName="ai-chat-body"
    headRight={
      <div className="ai-head-tools">
        <button
          className={`voice-toggle${speech.enabled ? " on" : ""}${speech.speaking ? " speaking" : ""}`}
          onClick={toggleSpeech}
          aria-label={speech.enabled ? "关闭语音陪伴" : "开启语音陪伴"}
          title={speech.enabled ? "关闭语音陪伴" : "开启语音陪伴"}
        >
          {speech.enabled ? <Volume2 size={15} /> : <VolumeX size={15} />}
        </button>
        <Sparkle size={16} color="var(--purple)" />
      </div>
    }
  >
    <div className="voice-status">
      <span className={speech.enabled ? "voice-dot on" : "voice-dot"} />
      {speech.enabled ? (speech.speaking ? "小愈正在轻声播报" : "语音陪伴已开启，AI 回复会自动朗读") : "语音陪伴已关闭，点右上"}
    </div>
    <div className="card ai-context-card">
      <div className="row between top">
        <div className="col">

```

```

    <span className="tiny">当前 Valence 状态</span>
    <span className="title">{current.level} • {current.score}</span>
  </div>
  <span className="chip teal">置信度 {current.confidence}%</span>
</div>
<span className="muted">{current.eegSummary}</span>
<div className="ai-context-action">
  <span>AI 推荐: {currentIntervention.title}</span>
</div>
</div>
<div className="ai-actions">
  <button className="btn btn-ghost btn-sm grow" onClick={() => fireToast("已为你预约校心理中心 • 周四 15:0
0, 老师会主动联系你")}>
    <CalendarCheck size={14} /> 预约校心理中心
  </button>
  <button
    className="btn btn-sm grow"
    style={{ background: "var(--amber-soft)", color: "var(--joy-deep)" }}
    onClick={() => fireToast("已接通 24h 心理援助热线 • 你不是一个人")}
  >
    <PhoneCall size={14} /> 24h 援助热线
  </button>
</div>
<div className="chat-list">
  {msgs.map((message) => (
    <motion.div
      key={message.id}
      className={`bubble-row ${message.role}`}
      initial={{ opacity: 0, y: 8 }}
      animate={{ opacity: 1, y: 0 }}
      transition={{ duration: 0.24 }}
    >
      {message.role === "ai" && <div className="ai-avatar">愈</div>}
      <div className={`bubble ${message.role}`}>{message.text}</div>
    </motion.div>
  ))}
  <AnimatePresence>
    {typing && (
      <motion.div className="bubble-row ai" initial={{ opacity: 0 }} animate={{ opacity: 1 }}
      exit={{ opacity: 0 }}>
        <div className="ai-avatar">愈</div>
        <div className="bubble ai typing"><span /><span /><span /></div>
      </motion.div>
    )}
  </AnimatePresence>
  <div ref={endRef} />
</div>
<div className="chat-composer">
  <div className="quick-asks">
    {aiQuickAsks.map((question) => (
      <button key={question} className="chip" onClick={() => void send(question)} disabled={ty
ping}>{question}</button>
    ))}
  </div>
  <div className="chat-input">

```

```

      <input
        value={input}
        onChange={(event) => setInput(event.target.value)}
        onKeyDown={(event) => { if (event.key === "Enter") void send(input); }}
        placeholder="想说点什么，都可以慢慢讲..."
        disabled={typing}
      />
      <button className="chat-send" onClick={() => void send(input)} aria-label="发送" disabled={t
yping}>
        <Send size={16} color="#fff" />
      </button>
    </div>
    <div className="row chat-note" style={{ gap: 8, padding: "2px 2px 0" }}>
      <ShieldCheck size={13} color="var(--teal-deep)" style={{ flex: "none" }} />
      <span className="tiny">小愈是 AI 倾听陪伴，不替代专业诊疗。如有危机，请使用上方“援助热线”或联系现实中的可信支持。</s
    </div>
  </div>
  <AnimatePresence>
    {toast && (
      <motion.div className="toast" initial={{ opacity: 0, y: 16 }} animate={{ opacity: 1, y: 0
    }} exit={{ opacity: 0, y: 16 }}>
        {toast}
      </motion.div>
    )}
  </AnimatePresence>
</SubScreen>
);
}
/* ===== 14 文件: src/screens/sub/GrowthScreen.tsx 共 147 行 ===== */
import { useState } from "react";
import { AreaChart, Area, ResponsiveContainer } from "recharts";
import { Flame, CalendarCheck2, Lock, Sparkles, ShieldCheck, TrendingUp } from "lucide-react";
import SubScreen from "../../components/SubScreen";
import { Chip, Reveal } from "../../components/ui";
import { checkinDays, checkinInfo, badges, weeklyReport } from "../../data/mock";
const RISK_STYLE: Record<string, { c: string; bg: string; t: string }> = {
  低: { c: "var(--teal-deep)", bg: "var(--teal-soft)", t: "心理状态平稳，无需特别关注" },
  中: { c: "var(--joy-deep)", bg: "var(--amber-soft)", t: "近期波动略大，建议增加放松训练" },
  高: { c: "var(--stress)", bg: "var(--pink-soft)", t: "建议尽快预约校心理中心老师" },
};
function MiniLine({ data, color, id }: { data: number[]; color: string; id: string }) {
  const d = data.map((v, i) => ({ i, v }));
  return (
    <div style={{ width: "100%", height: 40 }}>
      <ResponsiveContainer>
        <AreaChart data={d} margin={{ top: 4, right: 0, bottom: 0, left: 0 }}>
          <defs>
            <linearGradient id={id} x1="0" y1="0" x2="0" y2="1">
              <stop offset="0%" stopColor={color} stopOpacity={0.32} />
              <stop offset="100%" stopColor={color} stopOpacity={0} />
            </linearGradient>
          </defs>
          <Area type="monotone" dataKey="v" stroke={color} strokeWidth={2.2} fill={`url(#${id})`} do
t={false} />
        </AreaChart>
      </ResponsiveContainer>
    </div>
  );
}

```

```

    </ResponsiveContainer>
  </div>
);
}
export default function GrowthScreen() {
  const [checkedToday, setCheckedToday] = useState(false);
  const set = new Set(checkinDays);
  if (checkedToday) set.add(checkinInfo.today);
  const cells = Array.from({ length: checkinInfo.totalDays }, (_, i) => i + 1);
  const r = RISK_STYLE[weeklyReport.risk];
  return (
    <SubScreen title="成长档案">
      { /* streak hero */ }
      <Reveal i={0}>
        <div className="card hero g-purple">
          <div className="row between">
            <span className="kicker" style={{ color: "rgba(255,255,255,.78)" }}>连续打卡</span>
            <Chip variant="glass"><Flame size={12} /> {checkinInfo.streak + (checkedToday ? 1 : 0)}
          天</Chip>
          </div>
          <div className="row" style={{ gap: 14, alignItems: "baseline" }}>
            <span className="metric num">{checkinInfo.monthDone + (checkedToday ? 1 : 0)}</span>
            <span className="muted on-90">{checkinInfo.monthLabel} 已打卡天数</span>
          </div>
          <span className="body on-90" style={{ fontSize: 12.5 }}>每一次「看见情绪」，都在让心理韧性更结实一点 </span>
        </div>
      </Reveal>
      { /* calendar */ }
      <Reveal i={1}>
        <div className="card">
          <div className="section-head">
            <span className="title">{checkinInfo.monthLabel} 打卡日历</span>
            <span className="tiny">绿色为已打卡</span>
          </div>
          <div className="calendar">
            {["一", "二", "三", "四", "五", "六", "日"].map((w) => <span key={w} className="cal-w">{w}</span>
n>))}
            { /* 简化：从周一开始排（示例 5/1 是周四 -> 前置 3 个空格） */ }
            {[0, 1, 2].map((k) => <span key={"pad" + k} />)}
            {cells.map((d) => {
              const on = set.has(d);
              const isToday = d === checkinInfo.today;
              return (
                <span key={d} className={`cal-d${on ? " on" : ""}${isToday ? " today" : ""}`}>{d}</s
pan>
              );
            })}
          </div>
          <button className="btn btn-primary btn-block" disabled={checkedToday} style={checkedToday
? { opacity: 0.6 } : undefined} onClick={() => setCheckedToday(true)}>
            <CalendarCheck2 size={16} /> {checkedToday ? "今日已打卡" : "完成今日心情打卡"}
          </button>
        </div>
      </Reveal>
      { /* badges */ }

```

```

        <ArrowLeft size={20} color="var(--t-primary)" />
      </button>
      <span className="sub-title" style={accent ? { color: accent } : undefined}>{title}</span>
      <div className="sub-head-right">{headRight}</div>
    </div>
    <div className={`sub-body${bodyClassName ? ` ${bodyClassName}` : ""}`>{children}</div>
  </div>
);
}
/* ===== 19 文件: src/components/ui.tsx 共 172 行 ===== */
/* eslint-disable react-refresh/only-export-components */
import { useEffect, useId, useRef, useState, type CSSProperties, type ReactNode } from "react";
import { motion } from "framer-motion";
import { ChevronRight } from "lucide-react";
/* ----- count-up hook ----- */
export function useCountUp(target: number, duration = 900, deps: unknown[] = []) {
  const [val, setVal] = useState(0);
  const raf = useRef<number | null>(null);
  useEffect(() => {
    const start = performance.now();
    const tick = (now: number) => {
      const p = Math.min(1, (now - start) / duration);
      const eased = 1 - Math.pow(1 - p, 3);
      setVal(Math.round(target * eased));
      if (p < 1) raf.current = requestAnimationFrame(tick);
    };
    raf.current = requestAnimationFrame(tick);
    return () => { if (raf.current) cancelAnimationFrame(raf.current); };
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [target, duration, ...deps]);
  return val;
}
/* ----- stagger reveal ----- */
export function Reveal({
  children, i = 0, className, style,
}: { children: ReactNode; i?: number; className?: string; style?: CSSProperties }) {
  return (
    <motion.div
      className={className}
      style={style}
      initial={{ opacity: 0, y: 16 }}
      animate={{ opacity: 1, y: 0 }}
      transition={{ duration: 0.36, delay: 0.03 + i * 0.055, ease: [0.22, 1, 0.36, 1] }}
    >
      {children}
    </motion.div>
  );
}
/* ----- chip ----- */
export function Chip({ children, variant = "" }: { children: ReactNode; variant?: string }) {
  return <span className={`chip ${variant}`}>{children}</span>;
}
/* ----- section header ----- */
export function SectionHeader({ title, link }: { title: string; link?: string }) {
  return (

```

```

    <div className="section-head">
      <span className="title">{title}</span>
      {link && (
        <span className="link">
          {link} <ChevronRight size={13} />
        </span>
      )}
    </div>
  );
}
/* ----- mini bar chart ----- */
export function Bars({ data, color, animate = true }: { data: number[]; color: string; animate?: boo
  lean }) {
  const max = Math.max(...data);
  return (
    <div className="bars">
      {data.map((v, idx) => {
        const h = Math.max(4, (v / max) * 20);
        const active = idx === data.length - 1;
        return (
          <motion.i
            key={idx}
            initial={animate ? { height: 4 } : false}
            animate={{ height: h }}
            transition={{ duration: 0.5, delay: 0.12 + idx * 0.045, ease: [0.22, 1, 0.36, 1] }}
            style={{ background: color, opacity: active ? 1 : 0.3 }}
          />
        );
      })}
    </div>
  );
}
/* ----- progress track ----- */
export function Track({ pct, color, onGlass = false }: { pct: number; color: string; onGlass?: boole
  an }) {
  return (
    <div className={`track ${onGlass ? "on-glass" : ""}`}>
      <motion.i
        initial={{ width: 0 }}
        animate={{ width: `${Math.min(100, Math.max(0, pct))}%` }}
        transition={{ duration: 0.85, ease: [0.22, 1, 0.36, 1] }}
        style={{ background: color }}
      />
    </div>
  );
}
/* ----- gauge ring (gradient stroke + glow) ----- */
export function GaugeRing({
  value, size = 168, stroke = 14,
  trackColor = "rgba(255,255,255,.26)",
  from = "#ffffff", to = "rgba(255,255,255,.78)",
  glow = true, children,
}: {
  value: number; size?: number; stroke?: number; trackColor?: string;
  from?: string; to?: string; glow?: boolean; children?: ReactNode;

```



```

}) {
  const gid = useId().replace(/:/g, "");
  const r = (size - stroke) / 2;
  const c = 2 * Math.PI * r;
  return (
    <div className="gauge" style={{ width: size, height: size }}>
      {glow && <span className="gauge-glow" />}
      <svg viewBox={`0 0 ${size} ${size}`}>
        <defs>
          <linearGradient id={`g${gid}`} x1="0" y1="0" x2="1" y2="1">
            <stop offset="0%" stopColor={from} />
            <stop offset="100%" stopColor={to} />
          </linearGradient>
        </defs>
        <circle cx={size / 2} cy={size / 2} r={r} fill="none" stroke={trackColor} strokeWidth={stroke} />
      </svg>
      <div className="gauge-center">{children}</div>
    </div>
  );
}

/* ----- metric mini card ----- */
export function MetricCard({
  label, value, suffix = "%", note, color, trend, i = 0,
}: { label: string; value: number; suffix?: string; note?: string; color: string; trend: number[]; i
?: number }) {
  const n = useCountUp(value, 850);
  return (
    <Reveal i={i} className="grow" style={{ display: "flex" }}>
      <div className="metric-card" style={{ ["--mc-color" as string]: color, width: "100%" }}>
        <div className="mc-top">
          <span className="muted" style={{ fontSize: 11.5, fontWeight: 600 }}>{label}</span>
        </div>
        <div className="mc-val">
          <span className="metric-sm num">{n}</span> {suffix}</span>
          {note && <span className="tiny" style={{ color, fontWeight: 700 }}>{note}</span>}
        </div>
        <Bars data={trend} color={color} />
      </div>
    </Reveal>
  );
}

/* ----- segmented control ----- */
export function Segmented({ items, value, onChange }: { items: { key: string; label: string }[]; value: string; onChange: (k: string) => void }) {
  const idx = Math.max(0, items.findIndex((it) => it.key === value));

```

```

return (
  <div className="segmented">
    <motion.span
      className="seg-pill"
      animate={{ left: `calc(${(idx / items.length) * 100}% + 4px)`, width: `calc(${100 / items.le
ngth}% - 6px)` }}
      transition={{ type: "spring", stiffness: 380, damping: 32 }}
    />
    {items.map((it) => (
      <button key={it.key} className={it.key === value ? "active" : ""} onClick={() => onChange(it
.key)}>
        {it.label}
      </button>
    ))}
  </div>
);
}
/* ===== 20 文件: src/hooks/useClickSound.ts 共 88 行 ===== */
import { useEffect, useRef } from "react";
type AudioWindow = Window & typeof globalThis & {
  webkitAudioContext?: typeof AudioContext;
};
const INTERACTIVE_SELECTOR = [
  "button",
  "a",
  "[role='button']",
  "[data-click-sound]",
  "input[type='button']",
  "input[type='submit']",
  "input[type='reset']",
].join(",");
function isDisabledElement(element: Element) {
  return (
    element.hasAttribute("disabled") ||
    element.getAttribute("aria-disabled") === "true" ||
    element.classList.contains("disabled")
  );
}
function shouldPlayClickSound(event: PointerEvent) {
  if (event.button !== 0) return false;
  const target = event.target;
  if (!(target instanceof Element)) return false;
  const interactive = target.closest(INTERACTIVE_SELECTOR);
  if (!interactive || isDisabledElement(interactive)) return false;
  const tag = interactive.tagName.toLowerCase();
  const inputType = interactive instanceof HTMLInputElement ? interactive.type : "";
  const isTextField =
    tag === "textarea" ||
    tag === "select" ||
    (tag === "input" && ![ "button", "submit", "reset" ].includes(inputType));
  return !isTextField;
}
function playSoftClick(context: AudioContext, volume: number) {
  const now = context.currentTime;
  const master = context.createGain();

```

```

    master.gain.setValueAtTime(volume, now);
    master.connect(context.destination);
    const tap = context.createOscillator();
    const tapGain = context.createGain();
    tap.type = "sine";
    tap.frequency.setValueAtTime(760, now);
    tap.frequency.exponentialRampToValueAtTime(520, now + 0.045);
    tapGain.gain.setValueAtTime(0.0001, now);
    tapGain.gain.exponentialRampToValueAtTime(0.26, now + 0.006);
    tapGain.gain.exponentialRampToValueAtTime(0.0001, now + 0.052);
    tap.connect(tapGain).connect(master);
    const shine = context.createOscillator();
    const shineGain = context.createGain();
    shine.type = "triangle";
    shine.frequency.setValueAtTime(1280, now + 0.002);
    shine.frequency.exponentialRampToValueAtTime(930, now + 0.035);
    shineGain.gain.setValueAtTime(0.0001, now);
    shineGain.gain.exponentialRampToValueAtTime(0.075, now + 0.004);
    shineGain.gain.exponentialRampToValueAtTime(0.0001, now + 0.038);
    shine.connect(shineGain).connect(master);
    tap.start(now);
    shine.start(now + 0.002);
    tap.stop(now + 0.07);
    shine.stop(now + 0.055);
  }
export function useClickSound(volume = 0.18) {
  const audioRef = useRef<AudioContext | null>(null);
  const lastPlayedRef = useRef(0);
  useEffect(() => {
    const handlePointerDown = (event: PointerEvent) => {
      if (!shouldPlayClickSound(event)) return;
      const nowMs = performance.now();
      if (nowMs - lastPlayedRef.current < 42) return;
      lastPlayedRef.current = nowMs;
      const AudioCtor = window.AudioContext || (window as AudioWindow).webkitAudioContext;
      if (!AudioCtor) return;
      const context = audioRef.current ?? new AudioCtor();
      audioRef.current = context;
      if (context.state === "suspended") {
        void context.resume();
      }
      playSoftClick(context, volume);
    };
    window.addEventListener("pointerdown", handlePointerDown, { passive: true, capture: true });
    return () => {
      window.removeEventListener("pointerdown", handlePointerDown, { capture: true });
      void audioRef.current?.close();
      audioRef.current = null;
    };
  }, [volume]);
}
/* ===== 21 文件: src/hooks/useSpeechGuide.ts 共 74 行 ===== */
import { useCallback, useEffect, useRef, useState } from "react";
type SpeakOptions = {
  interrupt?: boolean;

```

```
};
function getChineseVoice() {
  const synth = window.speechSynthesis;
  const voices = synth.getVoices();
  return (
    voices.find((voice) => voice.lang.toLowerCase().startsWith("zh-cn")) ||
    voices.find((voice) => voice.lang.toLowerCase().startsWith("zh")) ||
    null
  );
}
export function useSpeechGuide(defaultEnabled = true) {
  const supported = typeof window !== "undefined" && "speechSynthesis" in window;
  const [enabled, setEnabled] = useState(defaultEnabled && supported);
  const [speaking, setSpeaking] = useState(false);
  const mountedRef = useRef(true);
  useEffect(() => {
    mountedRef.current = true;
    return () => {
      mountedRef.current = false;
      if (supported) window.speechSynthesis.cancel();
    };
  }, [supported]);
  const stop = useCallback(() => {
    if (!supported) return;
    window.speechSynthesis.cancel();
    setSpeaking(false);
  }, [supported]);
  const speak = useCallback((text: string, options: SpeakOptions = {}) => {
    if (!supported || !enabled) return false;
    const content = text.replace(/\s+/g, " ").trim();
    if (!content) return false;
    const synth = window.speechSynthesis;
    if (options.interrupt ?? true) synth.cancel();
    const utterance = new SpeechSynthesisUtterance(content);
    utterance.lang = "zh-CN";
    utterance.rate = 0.92;
    utterance.pitch = 1.02;
    utterance.volume = 0.9;
    const voice = getChineseVoice();
    if (voice) utterance.voice = voice;
    utterance.onstart = () => {
      if (mountedRef.current) setSpeaking(true);
    };
    utterance.onend = () => {
      if (mountedRef.current) setSpeaking(false);
    };
    utterance.onerror = () => {
      if (mountedRef.current) setSpeaking(false);
    };
    synth.speak(utterance);
    return true;
  }, [enabled, supported]);
  const toggle = useCallback(() => {
    if (!supported) return false;
    setEnabled((current) => {
```

```

        const next = !current;
        if (!next) window.speechSynthesis.cancel();
        return next;
    });
    return true;
}, [supported]);
return {
    enabled,
    setEnabled,
    supported,
    speaking,
    speak,
    stop,
    toggle,
};
}
/* ===== 22 文件: src/index.css 共 905 行 ===== */
/* =====
智愈莘莘 NeuroHeal 脑电情绪监测与干预反馈系统 V1.0 • 设计系统
tokens + base + components — 浅蓝 / 青绿 / 白, 少量紫点缀
===== */
:root {
    /* surfaces */
    --canvas: #f1f8fc;
    --surface: #ffffff;
    --surface-2: #fbfdfe;
    --blue-soft: #e8f5fc;
    --teal-soft: #e3f8f3;
    --purple-soft: #ecefbb;
    --amber-soft: #fcf2e3;
    --pink-soft: #fceef4;
    /* brand */
    --brand: #4fb6e6;
    --brand-2: #5cc4e8;
    --brand-deep: #237fb5;
    --teal: #2fc6ad;
    --teal-2: #45d2bb;
    --teal-deep: #1e9583;
    --purple: #7c6cf0;
    --purple-deep: #564ac0;
    /* mood */
    --calm: #2fc6ad;
    --focus: #3ba8e0;
    --stress: #e89a4a;
    --joy: #ecb83f;
    --joy-deep: #b9851d;
    /* ink */
    --t-primary: #16323f;
    --t-secondary: #5e7b8a;
    --t-tertiary: #92a8b6;
    --t-oncolor: #ffffff;
    --hairline: #e7f0f5;
    --hairline-2: #eef5f9;
    /* gradients */
    --grad-hero: linear-gradient(135deg, #57bce8 0%, #44c6b7 70%, #3fcaae 100%);

```

```

--grad-purple: linear-gradient(135deg, #8d7cf4 0%, #6e8df0 60%, #58a9e4 100%);
--grad-teal-blue: linear-gradient(135deg, #46c6c9 0%, #50b5e2 100%);
--grad-btn: linear-gradient(135deg, #54bce8 0%, #3fcaae 100%);
--grad-page:
  radial-gradient(60% 38% at 18% 8%, rgba(122, 196, 240, 0.28) 0%, rgba(122, 196, 240, 0) 60%),
  radial-gradient(50% 36% at 92% 18%, rgba(74, 207, 178, 0.24) 0%, rgba(74, 207, 178, 0) 62%),
  radial-gradient(60% 50% at 50% 100%, rgba(141, 124, 244, 0.14) 0%, rgba(141, 124, 244, 0) 60%),
  linear-gradient(180deg, #f3f9fd 0%, #eef6fa 50%, #eaf3f8 100%);
/* radius */
--r-card: 22px;
--r-hero: 26px;
--r-pill: 999px;
--r-sm: 14px;
--r-xs: 10px;
--r-phone: 46px;
/* shadows — layered & soft */
--shadow-xs: 0 1px 3px rgba(22, 50, 63, 0.05);
--shadow-soft: 0 2px 8px rgba(22, 50, 63, 0.04), 0 10px 22px -8px rgba(22, 50, 63, 0.10);
--shadow-card: 0 2px 8px rgba(22, 50, 63, 0.045), 0 14px 32px -10px rgba(22, 50, 63, 0.12);
--shadow-hero: 0 8px 18px rgba(40, 120, 160, 0.18), 0 24px 48px -12px rgba(40, 120, 160, 0.30);
--shadow-pop: 0 8px 20px rgba(22, 50, 63, 0.10), 0 2px 6px rgba(22, 50, 63, 0.06);
--shadow-phone:
  0 2px 8px rgba(22, 50, 63, 0.10),
  0 30px 60px -16px rgba(22, 50, 63, 0.24),
  0 56px 110px -28px rgba(34, 110, 150, 0.30);
--glow-brand: 0 10px 26px -6px rgba(79, 182, 230, 0.45);
--glow-teal: 0 10px 26px -6px rgba(47, 198, 173, 0.42);
--glow-purple: 0 10px 26px -6px rgba(124, 108, 240, 0.40);
--font: "Poppins", "PingFang SC", "Hiragino Sans GB", "Microsoft YaHei", system-ui, -apple-system,
  "Segoe UI", Roboto, sans-serif;
color-scheme: light;
}
* { margin: 0; padding: 0; box-sizing: border-box; }
html, body, #root { height: 100%; }
body {
  font-family: var(--font);
  color: var(--t-primary);
  background: var(--grad-page);
  background-attachment: fixed;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  text-rendering: optimizeLegibility;
  font-feature-settings: "ss01";
}
button { font-family: inherit; cursor: pointer; border: none; background: none; color: inherit; }
input, textarea { font-family: inherit; }
::-webkit-scrollbar { width: 0; height: 0; }
.num { font-variant-numeric: tabular-nums lining-nums; letter-spacing: 0.2px; }
/* ===== app shell ===== */
.app-shell {
  position: relative;
  min-height: 100%;
  display: flex;
  align-items: center;
  justify-content: center;

```

```

padding: 32px 16px;
gap: 56px;
flex-wrap: wrap;
overflow: hidden;
}
.app-shell::before, .app-shell::after {
  content: "";
  position: fixed;
  border-radius: 50%;
  filter: blur(70px);
  z-index: 0;
  pointer-events: none;
}
.app-shell::before { width: 420px; height: 420px; left: -120px; top: -80px; background: rgba(86, 196, 232, 0.30); }
.app-shell::after { width: 460px; height: 460px; right: -140px; bottom: -120px; background: rgba(56, 200, 174, 0.24); }
.app-shell > * { position: relative; z-index: 1; }
.app-aside { max-width: 264px; }
.app-aside .brand-row { display: flex; align-items: center; gap: 10px; }
.app-aside .brand-mark { width: 38px; height: 38px; border-radius: 12px; background: var(--grad-btn); display: flex; align-items: center; justify-content: center; color: #fff; box-shadow: var(--glow-brand); }
.app-aside h1 { font-size: 21px; font-weight: 700; color: var(--t-primary); letter-spacing: -0.3px; }
.app-aside .tag { display: inline-block; margin-top: 12px; font-size: 11.5px; font-weight: 600; color: var(--brand-deep); background: rgba(255, 255, 255, .7); border: 1px solid rgba(79, 182, 230, .25); padding: 6px 12px; border-radius: var(--r-pill); backdrop-filter: blur(6px); }
.app-aside p { margin-top: 14px; font-size: 13px; line-height: 1.7; color: var(--t-secondary); }
.app-aside ul { margin-top: 14px; list-style: none; font-size: 12.5px; line-height: 2.05; color: var(--t-secondary); }
.app-aside li { display: flex; align-items: center; gap: 8px; }
.app-aside li::before { content: ""; width: 6px; height: 6px; border-radius: 50%; background: var(--brand); flex: none; }
/* ===== phone frame ===== */
.phone-stage { position: relative; }
.phone-stage::before {
  content: ""; position: absolute; inset: -40px -30px -30px; border-radius: 60px;
  background: radial-gradient(50% 40% at 50% 25%, rgba(79, 182, 230, .30), transparent 70%),
    radial-gradient(46% 38% at 78% 78%, rgba(47, 198, 173, .24), transparent 70%);
  filter: blur(28px); z-index: -1;
}
.phone {
  position: relative;
  width: 378px;
  height: 818px;
  background: var(--surface);
  border-radius: var(--r-phone);
  box-shadow: var(--shadow-phone);
  overflow: hidden;
  display: flex;
  flex-direction: column;
  isolation: isolate;
}
.phone::after { /* glass edge highlight */

```

```

    content: ""; position: absolute; inset: 0; border-radius: var(--r-phone); pointer-events: none; z-
    index: 8;
    border: 1.5px solid rgba(255,255,255,.55);
    box-shadow: inset 0 0 0 1px rgba(22,50,63,.06), inset 0 1px 2px rgba(255,255,255,.7);
}
.phone-bg {
    position: absolute; inset: 0; z-index: 0;
    background:
        radial-gradient(70% 30% at 50% 0%, rgba(79,182,230,.10), transparent 60%),
        radial-gradient(60% 24% at 88% 6%, rgba(47,198,173,.10), transparent 60%),
        var(--canvas);
}
/* dynamic island */
.phone-island {
    position: absolute; top: 11px; left: 50%; transform: translateX(-50%);
    width: 108px; height: 30px; border-radius: 16px; background: #0e1d24; z-index: 9;
}
.phone-island::after { content: ""; position: absolute; right: 12px; top: 50%; transform: translateY
    (-50%); width: 8px; height: 8px; border-radius: 50%; background: #1a2c34; box-shadow: inset 0 0 3px
    rgba(120,200,230,.4); }
/* status bar */
.statusbar {
    position: relative; z-index: 5;
    height: 52px; flex: 0 0 52px;
    display: flex; align-items: center; justify-content: space-between;
    padding: 16px 26px 4px; font-size: 15px; font-weight: 600; color: var(--t-primary);
}
.statusbar .sb-right { display: flex; align-items: center; gap: 7px; }
.statusbar .sb-right .bat { display: flex; align-items: center; gap: 4px; font-size: 11px; color: va
    r(--t-secondary); }
/* scroll viewport */
.viewport { position: relative; z-index: 2; flex: 1 1 auto; overflow-y: auto; overflow-x: hidden; -w
    ebkit-overflow-scrolling: touch; }
.screen { padding: 6px 18px 30px; display: flex; flex-direction: column; gap: 15px; }
/* home indicator */
.home-indicator { position: absolute; left: 0; right: 0; bottom: 8px; display: flex; justify-content
    : center; z-index: 7; pointer-events: none; }
.home-indicator span { width: 134px; height: 5px; border-radius: 3px; background: rgba(22,50,63,.78)
    ; }
/* ===== bottom nav ===== */
.bottom-nav {
    position: relative; z-index: 7;
    flex: 0 0 86px; height: 86px;
    background: rgba(255,255,255,.92);
    backdrop-filter: blur(14px) saturate(1.2);
    border-top: 1px solid var(--hairline);
    box-shadow: 0 -6px 22px rgba(22,50,63,.07);
    display: flex; align-items: flex-start; padding: 9px 8px 22px;
}
.nav-tab { flex: 1; display: flex; flex-direction: column; align-items: center; gap: 4px; padding-to
    p: 3px; position: relative; }
.nav-tab .nav-icon-wrap {
    width: 40px; height: 32px; display: flex; align-items: center; justify-content: center; border-rad
    ius: var(--r-sm);
    transition: background .28s ease, box-shadow .28s ease;

```



```

}
.nav-tab.active .nav-icon-wrap { background: linear-gradient(135deg, rgba(79,182,230,.18), rgba(47,1
  98,173,.16)); box-shadow: inset 0 0 0 1px rgba(79,182,230,.18); }
.nav-tab svg { color: var(--t-tertiary); transition: color .28s ease; }
.nav-tab.active svg { color: var(--brand); }
.nav-tab .nav-label { font-size: 10px; font-weight: 600; color: var(--t-tertiary); transition: color
  .28s ease; letter-spacing: .2px; }
.nav-tab.active .nav-label { color: var(--brand-deep); }
.nav-tab .nav-indic { position: absolute; top: -5px; width: 18px; height: 3px; border-radius: 2px; b
  ackground: var(--grad-btn); }
/* ===== atoms ===== */
.card {
  background: var(--surface);
  border-radius: var(--r-card);
  padding: 16px;
  box-shadow: var(--shadow-card);
  display: flex; flex-direction: column; gap: 12px;
  position: relative;
}
.card.flat { box-shadow: none; }
.card.soft { box-shadow: var(--shadow-soft); }
.card.outline { box-shadow: var(--shadow-xs); border: 1px solid var(--hairline); }
.card.hero {
  border-radius: var(--r-hero); padding: 22px; gap: 16px; color: var(--t-oncolor); box-shadow: var(--
    shadow-hero);
  overflow: hidden; isolation: isolate;
}
.card.hero.g-hero { background: var(--grad-hero); }
.card.hero.g-purple { background: var(--grad-purple); }
.card.hero.g-teal-blue { background: var(--grad-teal-blue); }
/* hero glossy highlight + soft orb */
.card.hero::before {
  content: ""; position: absolute; z-index: -1; inset: -40% 30% auto auto; width: 220px; height: 220
    px; border-radius: 50%;
  background: radial-gradient(circle, rgba(255,255,255,.45), transparent 65%);
}
.card.hero::after {
  content: ""; position: absolute; z-index: -1; right: -50px; bottom: -70px; width: 200px; height: 2
    00px; border-radius: 50%;
  background: radial-gradient(circle, rgba(255,255,255,.18), transparent 62%);
}
.card.tint-blue { background: linear-gradient(160deg, #eef7fd, #e6f3fb); box-shadow: var(--shadow-xs
  ); border: 1px solid rgba(79,182,230,.16); }
.card.tint-teal { background: linear-gradient(160deg, #eafaf6, #elf6f1); box-shadow: var(--shadow-xs
  ); border: 1px solid rgba(47,198,173,.16); }
.card.tint-amber { background: linear-gradient(160deg, #fdf6ec, #fcf0e0); box-shadow: var(--shadow-x
  s); border: 1px solid rgba(232,154,74,.18); }
.card.tint-purple { background: linear-gradient(160deg, #f1effc, #ebe9fa); box-shadow: var(--shadow-
  xs); border: 1px solid rgba(124,108,240,.16); }
.row { display: flex; align-items: center; gap: 12px; }
.row.between { justify-content: space-between; }
.row.top { align-items: flex-start; }
.col { display: flex; flex-direction: column; gap: 4px; }
.grow { flex: 1; min-width: 0; }
.spacer { flex: 1; }

```

```

.kicker { font-size: 10.5px; font-weight: 700; letter-spacing: 1.4px; text-transform: uppercase; color: var(--t-tertiary); }
.eyebrow { font-size: 13px; font-weight: 600; letter-spacing: .2px; }
.h1 { font-size: 21px; font-weight: 700; letter-spacing: -.4px; color: var(--t-primary); }
.h2 { font-size: 18px; font-weight: 700; letter-spacing: -.2px; }
.title { font-size: 15.5px; font-weight: 700; color: var(--t-primary); letter-spacing: -.1px; }
.body { font-size: 13.5px; line-height: 1.55; color: var(--t-primary); }
.muted { font-size: 12.5px; line-height: 1.55; color: var(--t-secondary); }
.tiny { font-size: 11px; line-height: 1.5; color: var(--t-tertiary); }
.metric { font-size: 32px; font-weight: 700; line-height: 1; letter-spacing: -1px; }
.metric-sm { font-size: 22px; font-weight: 700; letter-spacing: -.5px; }
.on-90 { color: rgba(255,255,255,.94); }
.on-80 { color: rgba(255,255,255,.8); }
.on-70 { color: rgba(255,255,255,.72); }
.section-head { display: flex; align-items: center; justify-content: space-between; }
.section-head .title { font-size: 15px; }
.section-head .link { font-size: 11px; font-weight: 700; color: var(--brand-deep); display: flex; align-items: center; gap: 2px; }
.section-head button.link { background: transparent; }
.chip { display: inline-flex; align-items: center; gap: 5px; padding: 5px 11px; border-radius: var(--r-pill); font-size: 11px; font-weight: 700; background: var(--blue-soft); color: var(--brand-deep); white-space: nowrap; letter-spacing: .1px; }
.chip svg { flex: none; }
.chip.glass { background: rgba(255,255,255,.22); color: #fff; backdrop-filter: blur(6px); box-shadow: inset 0 0 0 1px rgba(255,255,255,.18); }
.chip.teal { background: var(--teal-soft); color: var(--teal-deep); }
.chip.amber { background: var(--amber-soft); color: var(--joy-deep); }
.chip.purple { background: var(--purple-soft); color: var(--purple-deep); }
.chip.solid-blue { background: var(--grad-btn); color: #fff; box-shadow: var(--glow-brand); }
.chip.live::before { content: ""; width: 6px; height: 6px; border-radius: 50%; background: #fff; box-shadow: 0 0 0 0 rgba(255,255,255,.5); animation: pulse 1.8s infinite; }
@keyframes pulse { 0% { box-shadow: 0 0 0 0 rgba(255,255,255,.55); } 70% { box-shadow: 0 0 0 7px rgba(255,255,255,0); } 100% { box-shadow: 0 0 0 0 rgba(255,255,255,0); } }
.btn { display: inline-flex; align-items: center; justify-content: center; gap: 6px; padding: 13px 24px; border-radius: var(--r-pill); font-size: 14px; font-weight: 700; transition: transform .12s ease, filter .15s ease, box-shadow .2s ease; letter-spacing: .1px; }
.btn:active { transform: scale(.975); }
.btn-primary { background: var(--grad-btn); color: #fff; box-shadow: var(--glow-brand); }
.btn-primary:hover { filter: brightness(1.04); box-shadow: 0 14px 30px -6px rgba(79,182,230,.5); }
.btn-ghost { background: var(--surface); color: var(--brand-deep); box-shadow: inset 0 0 0 1.5px rgba(79,182,230,.5); }
.btn-ghost:hover { background: var(--blue-soft); }
.btn-white { background: #fff; color: var(--purple-deep); box-shadow: 0 8px 20px -6px rgba(0,0,0,.18); }
.btn-outline-white { background: rgba(255,255,255,.12); color: #fff; box-shadow: inset 0 0 0 1.5px rgba(255,255,255,.7); }
.btn-block { width: 100%; }
.btn-sm { padding: 8px 16px; font-size: 12px; }
.avatar { border-radius: var(--r-pill); display: flex; align-items: center; justify-content: center; flex: none; overflow: hidden; background: var(--blue-soft); font-size: 18px; line-height: 1; }
.avatar.ring { box-shadow: 0 0 0 2px #fff, 0 0 0 3.5px rgba(79,182,230,.35); }
.dot { width: 7px; height: 7px; border-radius: var(--r-pill); flex: none; }
/* progress */
.track { height: 8px; border-radius: 4px; background: #e9f1f6; overflow: hidden; }
.track.on-glass { background: rgba(255,255,255,.26); }

```

```

.track > i { display: block; height: 100%; border-radius: 4px; box-shadow: 0 1px 4px rgba(0,0,0,.08)
; }
.bars { display: flex; align-items: flex-end; gap: 3px; height: 20px; }
.bars > i { width: 5px; border-radius: 2.5px; }
/* gauge */
.gauge { position: relative; display: flex; align-items: center; justify-content: center; }
.gauge svg { position: absolute; inset: 0; transform: rotate(-90deg); }
.gauge .gauge-glow { position: absolute; width: 70%; height: 70%; border-radius: 50%; background: radial-gradient(circle, rgba(255,255,255,.35), transparent 70%); }
.gauge .gauge-center { position: relative; display: flex; flex-direction: column; align-items: center; text-align: center; gap: 1px; }
/* metric mini card */
.metric-card { background: var(--surface); border-radius: var(--r-card); padding: 14px 13px; box-shadow: var(--shadow-soft); display: flex; flex-direction: column; gap: 8px; flex: 1; min-width: 0; position: relative; overflow: hidden; }
.metric-card::before { content: ""; position: absolute; left: 0; top: 0; bottom: 0; width: 3px; border-radius: 3px; background: var(--mc-color, var(--brand)); }
.metric-card .mc-top { display: flex; align-items: center; gap: 6px; }
.metric-card .mc-val { display: flex; align-items: baseline; gap: 4px; }
/* list row */
.list-row { display: flex; align-items: center; gap: 12px; padding: 11px 0; }
.list-row + .list-row { border-top: 1px solid var(--hairline-2); }
.icon-badge { width: 40px; height: 40px; border-radius: var(--r-sm); display: flex; align-items: center; justify-content: center; flex: none; }
.icon-badge.shadow { box-shadow: var(--shadow-xs); }
/* segmented */
.segmented { display: flex; background: #e4eef4; border-radius: 15px; padding: 4px; gap: 4px; position: relative; box-shadow: inset 0 1px 3px rgba(22,50,63,.06); }
.segmented button { flex: 1; padding: 9px 8px; border-radius: 12px; font-size: 12.5px; font-weight: 700; color: var(--t-secondary); transition: color .2s; position: relative; z-index: 1; }
.segmented button.active { color: var(--brand-deep); }
.segmented .seg-pill { position: absolute; top: 4px; bottom: 4px; background: #fff; border-radius: 12px; box-shadow: 0 2px 8px rgba(22,50,63,.14); z-index: 0; }
/* charts */
.chart-box { width: 100%; height: 168px; }
.chart-box.sm { height: 58px; }
.recharts-cartesian-grid line { stroke: var(--hairline); }
.recharts-default-tooltip { border-radius: 12px !important; border: 1px solid var(--hairline) !important; box-shadow: var(--shadow-pop) !important; font-size: 11.5px !important; padding: 8px 12px !important; }
.recharts-tooltip-label { color: var(--t-secondary) !important; font-size: 10.5px !important; margin-bottom: 4px !important; }
.recharts-tooltip-item { padding: 1px 0 !important; }
.legend-row { display: flex; gap: 14px; flex-wrap: wrap; }
.legend-row > span { display: inline-flex; align-items: center; gap: 6px; font-size: 11.5px; color: var(--t-secondary); font-weight: 600; }
/* misc */
.divider-v { width: 1px; align-self: stretch; background: var(--hairline); margin: 2px 0; }
.pill-row { display: flex; gap: 8px; flex-wrap: wrap; }
.input-fake { flex: 1; font-size: 12.5px; color: var(--t-tertiary); }
.sub-icon-btn { width: 34px; height: 34px; border-radius: 12px; display: flex; align-items: center; justify-content: center; background: var(--blue-soft); }
.stat-trip { display: flex; align-items: stretch; text-align: center; }
.stat-trip > .col { flex: 1; align-items: center; gap: 2px; }
/* race track */

```

```

.race-track { position: relative; height: 88px; border-radius: 18px; overflow: hidden; background: linear-gradient(180deg, #e9f5fc 0%, #def0f8 100%); box-shadow: inset 0 0 0 1px rgba(79,182,230,.16); }
.race-track.lane { position: absolute; left: 0; right: 0; height: 2px; background: repeating-linear-gradient(90deg, rgba(255,255,255,.85) 0 14px, transparent 14px 26px); }
.race-track.speed-lines { position: absolute; inset: 0; background: repeating-linear-gradient(90deg, transparent 0 36px, rgba(255,255,255,.4) 36px 38px); opacity: .5; }
.race-car { position: absolute; bottom: 12px; font-size: 26px; filter: drop-shadow(0 4px 6px rgba(0,0,.18)); }
.race-screen { background: radial-gradient(70% 40% at 50% 12%, #e7f8fd, transparent 65%), linear-gradient(180deg, #eef8fb 0%, #e8f3f4 100%); }
.race-body { flex: 1; overflow-y: auto; padding: 14px 18px 30px; display: flex; flex-direction: column; gap: 14px; }
.race-status-card,
.race-panel,
.race-countdown,
.race-control-panel,
.race-summary { gap: 14px; }
.race-status-card { position: relative; overflow: hidden; background: linear-gradient(155deg, rgba(255,255,255,.98), rgba(234,248,253,.94)); }
.race-status-card::after { content: ""; position: absolute; inset: auto -40px -90px auto; width: 190px; height: 190px; border-radius: 50%; background: radial-gradient(circle, rgba(79,182,230,.18), transparent 70%); pointer-events: none; }
.race-headline { position: relative; z-index: 1; }
.race-hud-stack { display: flex; flex-direction: column; align-items: flex-end; gap: 6px; }
.race-count-pill { display: inline-flex; align-items: center; gap: 6px; padding: 8px 12px; border-radius: var(--r-pill); background: var(--blue-soft); color: var(--brand-deep); font-size: 12px; font-weight: 700; }
.race-rush-pill { display: inline-flex; align-items: center; justify-content: center; min-width: 54px; padding: 5px 10px; border-radius: var(--r-pill); background: #eef4f7; color: var(--t-secondary); font-size: 11px; font-weight: 700; }
.race-rush-pill.active { background: var(--amber-soft); color: var(--joy-deep); box-shadow: 0 8px 18px -10px rgba(245,176,63,.8); }
.race-track.live { height: 150px; background: linear-gradient(180deg, #eaf8fc 0%, #dff1f7 100%); border: 1px solid rgba(79,182,230,.16); }
.race-finish { position: absolute; right: 12px; bottom: 15px; color: var(--brand-deep); display: inline-flex; align-items: center; justify-content: center; }
.race-crowd { position: absolute; left: 14px; top: 12px; color: rgba(18,54,69,.44); font-size: 11px; font-weight: 700; letter-spacing: .6px; text-transform: uppercase; }
.race-rival { position: absolute; bottom: 70px; transform: translateX(-50%); font-size: 24px; filter: drop-shadow(0 4px 6px rgba(0,0,0,.14)); }
.race-car.live { bottom: 24px; transform: translateX(-50%); }
.race-car.live.turbo { filter: drop-shadow(0 5px 9px rgba(10,114,160,.28)); }
.race-burst { position: absolute; bottom: 28px; left: calc(50% - 18px); transform: translateX(-50%); font-size: 18px; animation: flicker .4s infinite alternate; }
@keyframes flicker { from { opacity: .55; transform: translate(-50%, 1px) scale(.92); } to { opacity: 1; transform: translate(-50%, -1px) scale(1.06); } }
.race-metrics { display: grid; grid-template-columns: repeat(2, minmax(0, 1fr)); gap: 10px; }
.race-metric { display: flex; flex-direction: column; gap: 3px; padding: 12px; border-radius: 14px; background: #fff; box-shadow: var(--shadow-soft); }
.race-metric.strong { font-size: 20px; color: var(--brand-deep); }
.race-drive-row { display: flex; align-items: center; gap: 10px; }
.race-focus-bar { height: 10px; border-radius: 999px; background: #e4eef4; overflow: hidden; }
.race-drive-row.race-focus-bar { flex: 1; }
.race-focus-bar i { display: block; height: 100%; border-radius: inherit; background: var(--grad-btn

```

```

    ); transition: width .12s linear; }
.race-streak-pill { flex: none; padding: 7px 10px; border-radius: var(--r-pill); background: rgba(79
, 182, 230, .12); color: var(--brand-deep); font-size: 11px; font-weight: 700; }
.race-countdown { align-items: center; text-align: center; padding: 28px 18px; }
.race-countdown-number { font-size: 54px; color: var(--brand-deep); line-height: 1; }
.race-hold-button { width: 100%; min-height: 142px; border-radius: 24px; background: linear-gradient
(145deg, rgba(79, 182, 230, .18), rgba(63, 201, 178, .26)); color: var(--brand-deep); display: flex; flex-
direction: column; align-items: center; justify-content: center; gap: 6px; font-size: 24px; font-wei-
ght: 700; box-shadow: inset 0 0 0 1px rgba(79, 182, 230, .22), 0 18px 38px -22px rgba(28, 112, 140, .45);
touch-action: none; user-select: none; transition: transform .16s ease, background .18s ease, box-sh
adow .18s ease; }
.race-hold-button small { font-size: 12px; font-weight: 600; color: var(--t-secondary); }
.race-hold-button:active,
.race-hold-button.active { transform: scale(.985); background: linear-gradient(145deg, rgba(79, 182, 2
30, .34), rgba(63, 201, 178, .5)); box-shadow: inset 0 0 0 1px rgba(79, 182, 230, .28), 0 22px 42px -22px r
gba(28, 112, 140, .55); }
.race-summary-grid { display: grid; grid-template-columns: repeat(3, minmax(0, 1fr)); gap: 10px; }
.race-summary-grid > div { display: flex; flex-direction: column; gap: 4px; padding: 12px 10px; bord
er-radius: 14px; background: var(--blue-soft); text-align: center; }
.race-summary-grid strong { color: var(--brand-deep); font-size: 18px; }
/* hero stats inline (e.g. relax/focus/stress on home hero) */
.hero-mini-stats { display: flex; gap: 8px; }
.hero-mini-stats > div { flex: 1; background: rgba(255, 255, 255, .18); border-radius: 14px; padding: 1
0px 10px; backdrop-filter: blur(6px); box-shadow: inset 0 0 0 1px rgba(255, 255, 255, .16); }
@media (max-width: 820px) { .app-aside { display: none; } .app-shell { padding: 20px 10px; gap: 0; }
.phone-stage::before { display: none; } }
/* =====
子页面 / 弹层 (设备 • 处方 • 播放器 • AI • 成长 • 科普)
===== */
@keyframes spin { to { transform: rotate(360deg); } }
.spin { animation: spin 1s linear infinite; }
.phone-overlay {
position: absolute; left: 0; right: 0; top: 52px; bottom: 0; z-index: 12;
background: var(--canvas);
display: flex; flex-direction: column; overflow: hidden;
box-shadow: -12px 0 28px rgba(22, 50, 63, .10);
}
.sub-screen { display: flex; flex-direction: column; height: 100%; background: var(--canvas); }
.sub-header {
flex: 0 0 auto; display: flex; align-items: center; gap: 10px; padding: 12px 16px 12px 12px;
background: rgba(255, 255, 255, .86); backdrop-filter: blur(12px); border-bottom: 1px solid var(--hai
rline);
position: relative; z-index: 2;
}
.sub-back { width: 36px; height: 36px; border-radius: 12px; display: flex; align-items: center; just
ify-content: center; transition: background .15s; }
.sub-back:hover { background: var(--blue-soft); }
.sub-title { font-size: 16px; font-weight: 700; color: var(--t-primary); letter-spacing: -.2px; flex
: 1; }
.sub-head-right { width: 36px; display: flex; align-items: center; justify-content: center; }
.sub-body { flex: 1; overflow-y: auto; overflow-x: hidden; padding: 14px 18px 30px; display: flex; f
lex-direction: column; gap: 14px; }
/* ---- 播放器 ---- */
.player-screen { background: radial-gradient(70% 40% at 50% 12%, #e6f6fc, transparent 65%), linear-g
radient(180deg, #eef7fb 0%, #e7f3f2 100%); }

```

```

.player-body { flex: 1; display: flex; flex-direction: column; align-items: center; gap: 18px; padding: 18px 24px 30px; overflow-y: auto; }
.breath-wrap { position: relative; width: 240px; height: 240px; display: flex; align-items: center; justify-content: center; margin: 6px 0; }
.breath-ring { position: absolute; border-radius: 50%; }
.breath-ring.r1 { width: 220px; height: 220px; background: radial-gradient(circle, rgba(91,184,229,.30), rgba(63,201,178,.18)); filter: blur(2px); }
.breath-ring.r2 { width: 200px; height: 200px; background: rgba(255,255,255,.5); box-shadow: inset 0 0 30px rgba(91,184,229,.25); }
.breath-core { width: 180px; height: 180px; border-radius: 50%; background: linear-gradient(150deg, #65c4ec, #3fcaae); display: flex; align-items: center; justify-content: center; box-shadow: 0 14px 40px -8px rgba(63,180,200,.5), inset 0 2px 10px rgba(255,255,255,.5); }
.breath-label { color: #fff; font-size: 22px; font-weight: 700; letter-spacing: 4px; text-shadow: 0 1px 6px rgba(0,0,0,.12); }
.player-stats { display: flex; align-items: stretch; gap: 14px; width: 100%; max-width: 320px; background: rgba(255,255,255,.7); border-radius: 18px; padding: 12px 8px; box-shadow: var(--shadow-soft); }
.player-stats > .col { flex: 1; }
.player-btn { display: flex; align-items: center; justify-content: center; border-radius: 50%; transition: transform .12s; }
.player-btn.active { transform: scale(.94); }
.player-btn.main { width: 64px; height: 64px; background: var(--grad-btn); color: #fff; box-shadow: var(--glow-brand); }
.player-btn.ghost { width: 46px; height: 46px; background: rgba(255,255,255,.8); color: var(--brand-deep); box-shadow: var(--shadow-soft); }
.player-done-mask { position: absolute; inset: 0; z-index: 20; background: rgba(22,50,63,.35); backdrop-filter: blur(4px); display: flex; align-items: center; justify-content: center; padding: 24px; }
.player-done { align-items: center; text-align: center; gap: 12px; max-width: 300px; padding: 24px; }

/* ---- AI 对话 ---- */
.ai-head-tools { display: flex; align-items: center; gap: 10px; }
.voice-toggle { width: 30px; height: 30px; border-radius: 50%; display: flex; align-items: center; justify-content: center; color: var(--t-tertiary); background: #fff; box-shadow: var(--shadow-xs); transition: transform .18s, color .18s, background .18s, box-shadow .18s; }
.voice-toggle.on { color: var(--brand-deep); background: var(--blue-soft); box-shadow: 0 0 0 1px rgba(72,183,224,.18), var(--shadow-xs); }
.voice-toggle.speaking { animation: voicePulse 1.2s ease-in-out infinite; }
.voice-toggle.active { transform: scale(.94); }
.voice-status { flex: 0 0 auto; display: flex; align-items: center; gap: 7px; padding: 9px 12px; border-radius: var(--r-pill); background: rgba(255,255,255,.72); color: var(--t-secondary); font-size: 11.5px; font-weight: 700; box-shadow: var(--shadow-xs); }
.voice-dot { width: 7px; height: 7px; border-radius: 50%; background: var(--t-tertiary); flex: none; }
.voice-dot.on { background: var(--teal); box-shadow: 0 0 0 4px rgba(47,198,173,.14); }
@keyframes voicePulse { 0%, 100% { box-shadow: 0 0 0 1px rgba(72,183,224,.18), var(--shadow-xs); } 50% { box-shadow: 0 0 0 7px rgba(72,183,224,.13), var(--shadow-soft); } }
.ai-actions { display: flex; gap: 10px; }
.ai-actions .btn { box-shadow: none; }
.ai-chat-body { min-height: 0; overflow: hidden; padding-bottom: 18px; }
.ai-chat-body .ai-actions { flex: 0 0 auto; }
.chat-list { flex: 1 1 auto; min-height: 0; overflow-y: auto; overflow-x: hidden; display: flex; flex-direction: column; gap: 12px; padding: 4px 0 8px; }
.bubble-row { display: flex; gap: 8px; align-items: flex-end; }
.bubble-row.me { flex-direction: row-reverse; }
.ai-avatar { width: 30px; height: 30px; border-radius: 50%; background: var(--purple-soft); display:

```

```

    flex; align-items: center; justify-content: center; font-size: 16px; flex: none; box-shadow: var(--shadow-xs); }
.bubble { max-width: 78%; padding: 11px 14px; font-size: 13.5px; line-height: 1.6; border-radius: 16px; }
.bubble.ai { background: #fff; color: var(--t-primary); border-bottom-left-radius: 6px; box-shadow: var(--shadow-soft); }
.bubble.me { background: var(--grad-btn); color: #fff; border-bottom-right-radius: 6px; box-shadow: var(--glow-brand); }
.bubble.typing { display: flex; gap: 4px; padding: 14px; }
.bubble.typing span { width: 7px; height: 7px; border-radius: 50%; background: var(--t-tertiary); animation: blink 1.2s infinite; }
.bubble.typing span:nth-child(2) { animation-delay: .2s; }
.bubble.typing span:nth-child(3) { animation-delay: .4s; }
@keyframes blink { 0%, 60%, 100% { opacity: .25; transform: translateY(0); } 30% { opacity: 1; transform: translateY(-3px); } }
.quick-asks { display: flex; gap: 8px; overflow-x: auto; padding: 2px 0; }
.quick-asks.chip { flex: none; cursor: pointer; }
.quick-asks.chip:disabled,
.chat-send:disabled { opacity: .58; cursor: not-allowed; }
.chat-composer { flex: 0 0 auto; display: flex; flex-direction: column; gap: 12px; padding-top: 2px; background: linear-gradient(180deg, rgba(241,248,252,0), var(--canvas) 18%); }
.chat-input { display: flex; align-items: center; gap: 8px; background: #fff; border-radius: var(--r-pill); padding: 6px 6px 6px 16px; box-shadow: var(--shadow-soft); }
.chat-input input { flex: 1; border: none; outline: none; font-size: 13.5px; background: transparent; color: var(--t-primary); }
.chat-input input:disabled { opacity: .72; cursor: wait; }
.chat-input input::placeholder { color: var(--t-tertiary); }
.chat-send { width: 34px; height: 34px; border-radius: 50%; background: var(--grad-btn); display: flex; align-items: center; justify-content: center; flex: none; box-shadow: var(--glow-brand); }
.chat-note { flex: 0 0 auto; }
.toast { position: absolute; left: 50%; bottom: 24px; transform: translateX(-50%); background: rgba(22,50,63,.92); color: #fff; font-size: 12.5px; font-weight: 600; padding: 11px 16px; border-radius: 14px; box-shadow: var(--shadow-pop); max-width: 88%; text-align: center; z-index: 30; }
/* ---- 成长档案: 日历 / 勋章 / 周报 ---- */
.calendar { display: grid; grid-template-columns: repeat(7, 1fr); gap: 6px; }
.cal-w { text-align: center; font-size: 10.5px; font-weight: 700; color: var(--t-tertiary); padding-bottom: 2px; }
.cal-d { aspect-ratio: 1; display: flex; align-items: center; justify-content: center; font-size: 12px; font-weight: 600; color: var(--t-secondary); border-radius: 10px; background: #f3f8fb; }
.cal-d.on { background: linear-gradient(150deg, var(--teal-2), var(--teal)); color: #fff; box-shadow: 0 4px 10px -3px rgba(47,198,173,.5); }
.cal-d.today { box-shadow: 0 0 0 2px var(--brand); }
.cal-d.on.today { box-shadow: 0 0 0 2px var(--brand), 0 4px 10px -3px rgba(47,198,173,.5); }
.badge-grid { display: grid; grid-template-columns: repeat(4, 1fr); gap: 10px; }
.badge { display: flex; flex-direction: column; align-items: center; text-align: center; gap: 4px; padding: 10px 4px; border-radius: 14px; background: var(--blue-soft); }
.badge.locked { background: #f1f5f8; }
.badge-emoji { width: 40px; height: 40px; border-radius: 50%; background: #fff; display: flex; align-items: center; justify-content: center; font-size: 20px; box-shadow: var(--shadow-xs); }
.badge.locked.badge-emoji { background: #e7eef2; box-shadow: none; }
.badge-name { font-size: 10.5px; font-weight: 700; color: var(--t-primary); }
.badge.locked.badge-name { color: var(--t-tertiary); }
.badge-desc { font-size: 9px; line-height: 1.3; color: var(--t-tertiary); }
.notice-trigger { position: relative; overflow: visible; }
.notice-trigger > span {

```

```

position: absolute;
top: -8px;
right: -9px;
min-width: 22px;
height: 22px;
padding: 0 6px;
border-radius: 999px;
display: inline-flex;
align-items: center;
justify-content: center;
background: linear-gradient(135deg, #f29a45, #e9774e);
color: #fff;
font-size: 11px;
font-weight: 800;
line-height: 1;
border: 2px solid #fff;
box-shadow: 0 8px 18px rgba(232,154,74,.34);
animation: noticeBadgePop 1.8s ease-in-out infinite;
}

.notice-hero,
.notice-group { gap: 14px; }
.notice-hero { background: linear-gradient(150deg, rgba(255,255,255,.98), rgba(230,246,252,.94)); }
.notice-orb { width: 48px; height: 48px; border-radius: 18px; display: flex; align-items: center; justify-content: center; flex: none; color: var(--brand-deep); background: linear-gradient(145deg, rgba(79,182,230,.18), rgba(124,108,240,.16)); box-shadow: inset 0 0 0 1px rgba(79,182,230,.16); }
.notice-empty { display: flex; align-items: center; gap: 8px; padding: 12px; border-radius: 16px; background: #f7fbfd; color: var(--t-secondary); font-size: 12px; font-weight: 700; }
.notice-item { width: 100%; display: flex; align-items: flex-start; gap: 12px; padding: 12px 0; text-align: left; }
.notice-item + .notice-item { border-top: 1px solid var(--hairline-2); }
.notice-item.unread { padding: 12px; border-radius: 18px; background: linear-gradient(145deg, rgba(255,255,255,.98), rgba(233,247,252,.92)); box-shadow: inset 0 0 0 1px rgba(79,182,230,.12); }
.notice-item.unread + .notice-item.unread { border-top: none; margin-top: 8px; }
.notice-item.compact { align-items: center; }
.notice-title-row { gap: 8px; }
.notice-title-row strong,
.notice-item.compact strong { color: var(--t-primary); font-size: 13px; }
.notice-dot { width: 8px; height: 8px; border-radius: 999px; flex: none; background: var(--stress); box-shadow: 0 0 0 5px rgba(232,154,74,.12); }
@keyframes purchasePulse { 0%, 100% { transform: scaleX(.9); opacity: .7; } 50% { transform: scaleX(1.08); opacity: 1; } }
@keyframes purchaseBounce { from { transform: translateY(0) rotate(-2deg); } to { transform: translateY(-5px) rotate(2deg); } }
@keyframes noticeBadgePop { 0%, 100% { transform: scale(1); } 50% { transform: scale(1.08); } }
@media (max-width: 520px) {
  .export-choice-grid { grid-template-columns: 1fr; }
  .profile-extra-grid { grid-template-columns: 1fr; }
  .shop-wallet { grid-template-columns: 1fr 1fr; }
  .shop-wallet-divider { display: none; }
  .shop-wallet-badge { grid-column: 1 / -1; justify-self: start; }
  .wishlist-footer { align-items: flex-start; flex-direction: column; }
}

/* ---- 科普短视频 ---- */
.cat-row { display: flex; gap: 8px; overflow-x: auto; padding: 2px 0; }
.cat-chip { flex: none; padding: 7px 13px; border-radius: var(--r-pill); font-size: 12px; font-weight: 800; }

```



```

    t: 700; background: #eef4f8; color: var(--t-secondary); transition: all .2s; }
.cat-chip.active { background: var(--grad-btn); color: #fff; box-shadow: var(--glow-brand); }
.video-card { display: flex; gap: 12px; padding: 10px; background: #fff; border-radius: 16px; box-sh
    adow: var(--shadow-soft); width: 100%; text-align: left; }
.video-thumb { position: relative; width: 116px; height: 80px; border-radius: 12px; flex: none; disp
    lay: flex; align-items: center; justify-content: center; overflow: hidden; }
.video-thumb.tint-blue { background: linear-gradient(150deg, #d9eefb, #cfe6f7); }
.video-thumb.tint-teal { background: linear-gradient(150deg, #d6f3ec, #cdeee5); }
.video-thumb.tint-amber { background: linear-gradient(150deg, #f8ecd9, #f6e5cd); }
.video-thumb.tint-purple { background: linear-gradient(150deg, #e6e2f8, #ddd8f4); }
.video-emoji { font-size: 30px; }
.video-play { position: absolute; inset: 0; display: flex; align-items: center; justify-content: cen
    ter; }
.video-dur { position: absolute; right: 6px; bottom: 6px; background: rgba(22, 50, 63, .62); color: #ff
    f; font-size: 9.5px; font-weight: 700; padding: 2px 6px; border-radius: 6px; display: flex; align-it
    ems: center; gap: 3px; }
.video-thumb.big { width: 100%; height: 150px; border-radius: 16px; margin-bottom: 4px; }
.video-close { position: absolute; right: 10px; top: 10px; width: 28px; height: 28px; border-radius:
    50%; background: rgba(255, 255, 255, .85); display: flex; align-items: center; justify-content: center
    ; color: var(--t-primary); }
.video-modal { align-items: stretch; text-align: left; gap: 10px; max-width: 320px; padding: 14px; }
/* ===== EEG-Valence 闭环改版: 更干净、舒心、主线明确 ===== */
:root {
    --canvas: #f6fbfd;
    --surface: #ffffff;
    --surface-2: #fbfdfe;
    --grad-page: linear-gradient(180deg, #f7fbfd 0%, #eef7fb 100%);
    --shadow-card: 0 1px 2px rgba(22, 50, 63, 0.04), 0 12px 30px -18px rgba(22, 50, 63, 0.18);
    --shadow-hero: 0 2px 8px rgba(22, 50, 63, 0.06), 0 18px 40px -24px rgba(22, 50, 63, 0.20);
    --shadow-phone:
        0 2px 8px rgba(22, 50, 63, 0.08),
        0 28px 70px -34px rgba(34, 110, 150, 0.34);
}
body { background: var(--grad-page); }
.app-shell::before,
.app-shell::after,
.phone-stage::before,
.card.hero::before,
.card.hero::after { display: none; }
.app-shell { gap: 48px; }
.app-aside.tag { background: #fff; }
.card { border: 1px solid rgba(231, 240, 245, 0.78); }
.card.hero { background: #fff; color: var(--t-primary); }
.monitor-hero,
.intervention-hero,
.regulation-hero,
.record-hero,
.report-loop-hero {
    background: linear-gradient(180deg, #ffffff 0%, #f4fbfd 100%);
    border: 1px solid rgba(79, 182, 230, 0.15);
    box-shadow: var(--shadow-card);
}
.monitor-level {
    color: var(--brand-deep);
    font-size: 28px;

```

```
    font-weight: 800;
    letter-spacing: -0.4px;
  }
  .monitor-signal-grid,
  .before-after-grid,
  .intervention-stat-grid {
    display: grid;
    grid-template-columns: repeat(2, minmax(0, 1fr));
    gap: 10px;
  }
  .monitor-signal-grid > div,
  .before-after-grid > div,
  .intervention-stat-grid > div {
    min-height: 84px;
    display: flex;
    flex-direction: column;
    justify-content: space-between;
    gap: 6px;
    padding: 12px;
    border-radius: 18px;
    background: rgba(255, 255, 255, .82);
    box-shadow: inset 0 0 0 1px rgba(79, 182, 230, .12);
  }
  .monitor-signal-grid span,
  .before-after-grid span,
  .intervention-stat-grid span {
    color: var(--t-tertiary);
    font-size: 11px;
    font-weight: 700;
  }
  .monitor-signal-grid strong,
  .before-after-grid strong,
  .intervention-stat-grid strong {
    color: var(--brand-deep);
    font-size: 18px;
    line-height: 1.15;
  }
  .before-after-grid em {
    color: var(--t-secondary);
    font-size: 11px;
    font-style: normal;
    font-weight: 700;
  }
  .monitor-eeeg-panel {
    display: flex;
    flex-direction: column;
    gap: 10px;
    padding: 13px;
    border-radius: 18px;
    background: #fff;
    box-shadow: inset 0 0 0 1px rgba(79, 182, 230, .12);
  }
  .ai-feedback-card {
    background: linear-gradient(180deg, #ffffff 0%, #f3fbf8 100%);
  }
```

```
.recommended-action {
  display: flex;
  align-items: center;
  gap: 12px;
  padding: 13px;
  border-radius: 18px;
  background: rgba(255,255,255,.9);
  box-shadow: inset 0 0 0 1px rgba(47,198,173,.18);
}

.recommended-action.highlight {
  background: linear-gradient(145deg, rgba(255,255,255,.96), rgba(230,248,244,.96));
}

.recommended-action strong {
  color: var(--t-primary);
  font-size: 14px;
}

.recommended-action small {
  color: var(--t-secondary);
  font-size: 11.5px;
  line-height: 1.35;
}

.loop-flow {
  display: grid;
  grid-template-columns: repeat(5, minmax(0, 1fr));
  gap: 6px;
}

.loop-flow span {
  min-height: 44px;
  display: inline-flex;
  align-items: center;
  justify-content: center;
  text-align: center;
  padding: 8px 5px;
  border-radius: 14px;
  background: #f4f8fb;
  color: var(--t-secondary);
  font-size: 10.5px;
  font-weight: 800;
  line-height: 1.25;
}

.loop-flow span.active {
  background: linear-gradient(135deg, rgba(79,182,230,.18), rgba(47,198,173,.20));
  color: var(--brand-deep);
  box-shadow: inset 0 0 0 1px rgba(79,182,230,.18);
}

.compact-loop-row .icon-badge { width: 36px; height: 36px; }
.report-loop-hero .ai-report-note,
.ai-report-note {
  display: flex;
  gap: 9px;
  align-items: flex-start;
  padding: 12px;
  border-radius: 16px;
  background: rgba(255,255,255,.82);
  color: var(--t-secondary);
}
```

```
    font-size: 12.5px;
    line-height: 1.55;
}
.loop-delta-pill,
.intervention-score {
    min-width: 54px;
    height: 54px;
    border-radius: 18px;
    display: inline-flex;
    align-items: center;
    justify-content: center;
    background: var(--teal-soft);
    color: var(--teal-deep);
    font-size: 22px;
    font-weight: 800;
}
.intervention-action-card {
    width: 100%;
    text-align: left;
}
.intervention-stat-grid {
    grid-template-columns: repeat(3, minmax(0, 1fr));
}
.regulation-hero .race-track,
.race-track.calm {
    background: linear-gradient(180deg, #eef8fc 0%, #e6f5f1 100%);
    box-shadow: inset 0 0 0 1px rgba(79, 182, 230, .14);
}
.record-hero .before-after-grid > div {
    background: #fff;
}
.ai-context-card {
    gap: 10px;
    background: linear-gradient(180deg, #ffffff 0%, #f4fbfd 100%);
    box-shadow: var(--shadow-xs);
}
.ai-context-action {
    padding: 10px 12px;
    border-radius: 14px;
    background: var(--teal-soft);
    color: var(--teal-deep);
    font-size: 12px;
    font-weight: 700;
}
/* ---- 记录页：闭环时间轴 / 成长概览 / 辅助入口 ---- */
.loop-timeline { display: flex; flex-direction: column; gap: 14px; }
.loop-timeline-row { display: flex; gap: 12px; align-items: flex-start; }
.loop-timeline-dot {
    width: 12px; height: 12px; border-radius: 50%; margin-top: 6px; flex: none;
    background: var(--teal); box-shadow: 0 0 0 4px rgba(47, 198, 173, .15);
}
.loop-timeline-dot[data-status="进行中"] {
    background: var(--brand); box-shadow: 0 0 0 4px rgba(79, 182, 230, .18);
    animation: pulse 1.8s infinite;
}
```

```

.loop-timeline-body { flex: 1; display: flex; flex-direction: column; gap: 6px; min-width: 0; }
.loop-timeline-body .chip { padding: 4px 9px; font-size: 10.5px; }
.loop-delta { font-size: 14px; font-weight: 700; letter-spacing: .2px; }
.loop-delta.up { color: var(--teal-deep); }
.loop-delta.down { color: var(--stress); }
.growth-grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 10px; }
.growth-stat {
  display: flex; flex-direction: column; align-items: flex-start; gap: 4px;
  padding: 12px; border-radius: 14px; background: var(--surface-2);
  box-shadow: inset 0 0 0 1px var(--hairline);
}
.growth-stat strong { font-size: 18px; color: var(--brand-deep); font-variant-numeric: tabular-nums;
}
.growth-stat span { font-size: 11px; color: var(--t-tertiary); }
.badge-strip { display: flex; gap: 8px; overflow-x: auto; padding: 2px 0; }
.badge-chip {
  flex: none; display: flex; flex-direction: column; align-items: center; gap: 4px;
  padding: 8px 12px; border-radius: 14px; background: var(--blue-soft);
  min-width: 64px;
}
.badge-emoji { font-size: 20px; line-height: 1; }
.aux-entry-row { display: grid; grid-template-columns: repeat(3, 1fr); gap: 10px; }
.aux-entry {
  display: flex; flex-direction: column; align-items: center; gap: 6px;
  padding: 12px 8px; border-radius: var(--r-card);
  background: var(--surface); box-shadow: var(--shadow-xs);
  border: 1px solid var(--hairline-2);
  transition: transform .15s ease, box-shadow .2s ease;
}
.aux-entry:hover { transform: translateY(-1px); box-shadow: var(--shadow-soft); }
.aux-entry-ico {
  width: 36px; height: 36px; border-radius: 12px;
  display: flex; align-items: center; justify-content: center;
}
@media (max-width: 520px) {
  .monitor-signal-grid,
  .before-after-grid { grid-template-columns: 1fr 1fr; }
  .loop-flow { grid-template-columns: repeat(5, minmax(48px, 1fr)); overflow-x: auto; }
  .recommended-action { align-items: stretch; flex-direction: column; }
  .recommended-action .btn { width: 100%; }
  .growth-grid { grid-template-columns: 1fr 1fr; }
}
/* ===== 23 文件: vite.config.ts 共 47 行 ===== */
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import { VitePWA } from 'vite-plugin-pwa'
import { viteSingleFile } from 'vite-plugin-singlefile'
// https://vite.dev/config/
// 默认 `vite build` -> 普通 PWA 构建 (dist/, 可安装 / 可部署)
// `vite build --mode single` -> 把整个 App 打进单个 index.html (distSingle/, 可直接拷给别人/手机打开)
export default defineConfig(({ mode }) => {
  const single = mode === 'single'
  return {
    plugins: [
      react(),

```

```

... (single
  ? [viteSingleFile()]
  : [
    VitePWA({
      registerType: 'autoUpdate',
      includeAssets: ['icon.svg'],
      manifest: {
        name: '智愈莘莘 NeuroHeal 脑电情绪监测与干预反馈系统',
        short_name: 'NeuroHeal',
        description: '基于 EEG 脑电信号的实时情绪识别与 AI 反馈干预闭环系统 (V1.0)',
        theme_color: '#4fb6e6',
        background_color: '#f1f8fc',
        display: 'standalone',
        orientation: 'portrait',
        start_url: '.',
        scope: '.',
        lang: 'zh-CN',
        icons: [
          { src: 'icon.svg', sizes: 'any', type: 'image/svg+xml', purpose: 'any' },
          { src: 'icon.svg', sizes: 'any', type: 'image/svg+xml', purpose: 'maskable' },
        ],
      },
      workbox: {
        globPatterns: ['**/*. {js,css,html,svg,png,woff2}'],
      },
    }),
  ],
],
server: {
  proxy: {
    '/api': 'http://127.0.0.1:8787',
  },
},
}
})
/* ===== 24 文件: eslint.config.js 共 21 行 ===== */
import js from '@eslint/js'
import globals from 'globals'
import reactHooks from 'eslint-plugin-react-hooks'
import reactRefresh from 'eslint-plugin-react-refresh'
import tseslint from 'typescript-eslint'
import { defineConfig, globalIgnores } from 'eslint/config'
export default defineConfig([
  globalIgnores(['dist']),
  {
    files: ['**/*. {ts,tsx}'],
    extends: [
      js.configs.recommended,
      tseslint.configs.recommended,
      reactHooks.configs.flat.recommended,
      reactRefresh.configs.vite,
    ],
    languageOptions: {
      globals: globals.browser,
    },
  },
])

```

```
    },
  ])
/* ===== 25 文件: tsconfig.json 共 7 行 ===== */
{
  "files": [],
  "references": [
    { "path": "./tsconfig.app.json" },
    { "path": "./tsconfig.node.json" }
  ]
}
/* ===== 26 文件: tsconfig.app.json 共 23 行 ===== */
{
  "compilerOptions": {
    "tsBuildInfoFile": "./node_modules/.tmp/tsconfig.app.tsbuildinfo",
    "target": "es2023",
    "lib": ["ES2023", "DOM"],
    "module": "esnext",
    "types": ["vite/client"],
    "skipLibCheck": true,
    /* Bundler mode */
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "verbatimModuleSyntax": true,
    "moduleDetection": "force",
    "noEmit": true,
    "jsx": "react-jsx",
    /* Linting */
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "erasableSyntaxOnly": true,
    "noFallthroughCasesInSwitch": true
  },
  "include": ["src"]
}
/* ===== 27 文件: tsconfig.node.json 共 22 行 ===== */
{
  "compilerOptions": {
    "tsBuildInfoFile": "./node_modules/.tmp/tsconfig.node.tsbuildinfo",
    "target": "es2023",
    "lib": ["ES2023"],
    "module": "esnext",
    "types": ["node"],
    "skipLibCheck": true,
    /* Bundler mode */
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "verbatimModuleSyntax": true,
    "moduleDetection": "force",
    "noEmit": true,
    /* Linting */
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "erasableSyntaxOnly": true,
    "noFallthroughCasesInSwitch": true
  },
}
```

```
"include": ["vite.config.ts"]
}
/* ===== 28 文件: package.json 共 41 行 ===== */
{
  "name": "neuroheal-app",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "api": "node --env-file-if-exists=.env server/deepseek.mjs",
    "build": "tsc -b && vite build",
    "build:single": "vite build --mode single --outDir distSingle --emptyOutDir",
    "lint": "eslint .",
    "preview": "vite preview",
    "start": "node --env-file-if-exists=.env server/deepseek.mjs"
  },
  "engines": {
    "node": ">=20.11.0"
  },
  "dependencies": {
    "framer-motion": "^12.38.0",
    "lucide-react": "^1.14.0",
    "react": "^19.2.6",
    "react-dom": "^19.2.6",
    "recharts": "^3.8.1"
  },
  "devDependencies": {
    "@eslint/js": "^10.0.1",
    "@types/node": "^24.12.3",
    "@types/react": "^19.2.14",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^6.0.1",
    "eslint": "^10.3.0",
    "eslint-plugin-react-hooks": "^7.1.1",
    "eslint-plugin-react-refresh": "^0.5.2",
    "globals": "^17.6.0",
    "typescript": "^6.0.2",
    "typescript-eslint": "^8.59.2",
    "vite": "^8.0.12",
    "vite-plugin-pwa": "^1.3.0",
    "vite-plugin-singlefile": "^2.3.3"
  }
}
/* ===== 29 文件: server/deepseek.mjs 共 183 行 ===== */
import { createReadStream, existsSync, statSync } from "node:fs";
import { extname, join, normalize, resolve } from "node:path";
import { createServer } from "node:http";
const PORT = Number(process.env.PORT || process.env.DEEPSEEK_PORT || 8787);
const HOST = process.env.HOST || process.env.DEEPSEEK_HOST || "0.0.0.0";
const BASE_URL = (process.env.DEEPSEEK_BASE_URL || "https://api.deepseek.com").replace(/\/\$/, "");
const MODEL = process.env.DEEPSEEK_MODEL || "deepseek-v4-flash";
const RAW_API_KEY = process.env.DEEPSEEK_API_KEY || "";
const API_KEY = RAW_API_KEY.trim();
const REQUEST_LIMIT = 1_000_000;
```



```

const MAX_HISTORY = 12;
const DIST_DIR = resolve(process.cwd(), "dist");
const SYSTEM_PROMPT = [
  "你是 NeuroHeal 应用里的 AI 心理陪伴助手“小愈”。",
  "系统核心场景是 EEG 脑电信号监测、Valence 情绪效价识别和 AI 反馈干预闭环。",
  "你的目标是基于用户当前状态进行温和倾听，帮助用户梳理感受，并给出简短、具体、可执行的调节建议。",
  "如果对话中包含以 [Valence 上下文] 开头的系统消息，请把它当作系统刚刚识别到的实时脑电状态，优先据此调整回复语气和建议：",
  " - 低效价 / 较低效价：先给予温和安抚，再建议进入呼吸训练、数字处方、意念赛车或 AI 陪伴；",
  " - 较高效价 / 高效价：肯定当前稳定状态，建议继续轻量专注训练或成长记录；",
  " - 当信号质量低于 70% 时可温和提醒佩戴位置；",
  " - 在回复里可以自然引用 Valence 等级（如“你现在处于较低效价”），但避免引用裸 score 数字。",
  "不要冒充医生，不要做诊断，不要给出确定性的医疗结论。",
  "如果用户提到自伤、自杀、伤害他人或明显危机，请先表达关切，并鼓励立即联系当地紧急支持、可信任的人或线下专业机构。",
  "回复默认使用中文，语气平静自然，长度控制在 2 到 5 句。",
].join("\n");
const CONTENT_TYPES = {
  ".css": "text/css; charset=utf-8",
  ".html": "text/html; charset=utf-8",
  ".ico": "image/x-icon",
  ".js": "text/javascript; charset=utf-8",
  ".json": "application/json; charset=utf-8",
  ".png": "image/png",
  ".svg": "image/svg+xml",
  ".webmanifest": "application/manifest+json",
  ".woff2": "font/woff2",
};
function sendJson(response, statusCode, payload) {
  response.writeHead(statusCode, {
    "Content-Type": "application/json; charset=utf-8",
    "Cache-Control": "no-store",
  });
  response.end(JSON.stringify(payload));
}
function isAsciiHeaderValue(value) {
  return /^[\x20-\x7E]+$/.test(value);
}
function readBody(request) {
  return new Promise((resolveBody, reject) => {
    let body = "";
    request.on("data", (chunk) => {
      body += chunk;
      if (body.length > REQUEST_LIMIT) {
        reject(new Error("Request body is too large."));
        request.destroy();
      }
    });
    request.on("end", () => resolveBody(body));
    request.on("error", reject);
  });
}
const ALLOWED_ROLES = new Set(["system", "assistant", "user"]);
const MAX_VALENCE_CONTEXT_LENGTH = 800;
function normalizeMessages(messages) {
  if (!Array.isArray(messages)) return [];
  return messages

```

```

    .filter((message) => message && ALLOWED_ROLES.has(message.role) && typeof message.content === "string")
    .map((message) => {
      const content = message.content.trim();
      if (message.role === "system") {
        return { role: "system", content: content.slice(0, MAX_VALENCE_CONTEXT_LENGTH) };
      }
      return { role: message.role, content };
    })
    .filter((message) => message.content.length > 0)
    .slice(-MAX_HISTORY);
}

async function handleChat(request, response) {
  if (!API_KEY) {
    sendJson(response, 500, { error: "Missing DEEPSEEK_API_KEY on the backend server." });
    return;
  }
  if (!isAsciiHeaderValue(API_KEY)) {
    sendJson(response, 500, {
      error: "DEEPSEEK_API_KEY contains non-ASCII characters. Re-enter the raw sk-* key only, without Chinese text, quotes, spaces, or line breaks.",
    });
    return;
  }
  try {
    const rawBody = await readBody(request);
    const payload = rawBody ? JSON.parse(rawBody) : {};
    const messages = normalizeMessages(payload.messages);
    if (!messages.some((message) => message.role === "user")) {
      sendJson(response, 400, { error: "At least one user message is required." });
      return;
    }
  }
  const upstream = await fetch(`${BASE_URL}/chat/completions`, {
    method: "POST",
    headers: {
      Authorization: `Bearer ${API_KEY}`,
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      model: MODEL,
      temperature: 0.7,
      max_tokens: 600,
      stream: false,
      messages: [
        { role: "system", content: SYSTEM_PROMPT },
        ...messages,
      ],
    }),
  });
  const data = await upstream.json();
  const reply = data?.choices?.[0]?.message?.content?.trim();
  if (!upstream.ok) {
    sendJson(response, upstream.status, { error: data?.error?.message || "DeepSeek request failed." });
    return;
  }

```

```

    }
    if (!reply) {
      sendJson(response, 502, { error: "DeepSeek returned an empty reply." });
      return;
    }
    sendJson(response, 200, { reply });
  } catch (error) {
    const message = error instanceof Error ? error.message : "Unexpected backend error.";
    sendJson(response, 500, { error: message });
  }
}

function safeAssetPath(pathname) {
  const decoded = decodeURIComponent(pathname);
  const relativePath = decoded === "/" ? "index.html" : decoded.replace(/^\/+/, "");
  const normalizedPath = normalize(relativePath).replace(/^(\\.\\. [\/\\])+/, "");
  const fullPath = resolve(DIST_DIR, normalizedPath);
  if (!fullPath.startsWith(DIST_DIR)) return null;
  return fullPath;
}

function serveStatic(request, response, pathname) {
  if (!existsSync(DIST_DIR)) {
    sendJson(response, 503, { error: "Frontend build not found. Run npm run build before starting production mode." });
    return;
  }

  const requestedPath = safeAssetPath(pathname);
  const assetPath = requestedPath && existsSync(requestedPath) && statSync(requestedPath).isFile()
    ? requestedPath
    : join(DIST_DIR, "index.html");
  const extension = extname(assetPath);
  const contentType = CONTENT_TYPES[extension] || "application/octet-stream";
  const cacheControl = assetPath.endsWith("index.html") ? "no-cache" : "public, max-age=31536000, immutable";

  response.writeHead(200, {
    "Content-Type": contentType,
    "Cache-Control": cacheControl,
  });
  createReadStream(assetPath).pipe(response);
}

const server = createServer((request, response) => {
  const url = new URL(request.url || "/", `http://${request.headers.host || `${HOST}:${PORT}`}`);
  if (request.method === "GET" && url.pathname === "/api/health") {
    sendJson(response, 200, {
      ok: true,
      model: MODEL,
      configured: Boolean(API_KEY),
      keyFormatValid: Boolean(API_KEY) && isAsciiHeaderValue(API_KEY),
    });
    return;
  }

  if (request.method === "POST" && url.pathname === "/api/chat") {
    void handleChat(request, response);
    return;
  }

  if (request.method === "GET" || request.method === "HEAD") {

```

```
    serveStatic(request, response, url.pathname);
    return;
  }
  sendJson(response, 404, { error: "Not found." });
});
server.listen(PORT, HOST, () => {
  console.log(`NeuroHeal server listening on http://${HOST}:${PORT}`);
});
```